

KUGGLE 2차 프로젝트

회귀분석을 통한 월마트의 주간 매출 예측

12기 문준영 . 12기 박슬옹 . 13기 김재환 . 13기 안시현 . 13기 이수아

CONTENTS

01

분석 개요

02

데이터 전처리
및 EDA

03

선형 회귀분석

04

비선형 회귀분석

05

결론

1. 분석 개요

분석 주제와 데이터 구성

“ 회귀분석을 통한
월마트의 주간 매출 예측 ”

Data : Walmart_Sales

<https://www.kaggle.com/datasets/mikhail1681/walmart-sales>

Columns

- Store: 월마트 매장 번호
- Date: 날짜
- Weekly_Sales: 주간 판매량
- Holiday_Flag: 휴일이 있는 주
휴일이 있는 주면 1, 아니면 0
- Temperature: 기온(화씨)
- Fuel_Price: 유가
- CPI: 소비자 물가지수
- Unemployment: 실업률

2. 데이터 전처리 및 EDA

결측치 파악, column 별 값의 개수

```
# missing
print("\n=== MISSING VALUES ===")
print(df.isna().sum())

# unique counts
print("\n=== UNIQUE COUNTS ===")
for c in df.columns:
    print(c, df[c].nunique())
```

```
=== MISSING VALUES ===
Store      0
Date       0
Weekly_Sales  0
Holiday_Flag  0
Temperature  0
Fuel_Price  0
CPI         0
Unemployment  0
dtype: int64
```

```
=== UNIQUE COUNTS ===
Store 45
Date 143
Weekly_Sales 6435
Holiday_Flag 2
Temperature 3528
Fuel_Price 892
CPI 2145
Unemployment 349
```

결측치가 없으므로 제거 과정 없이 분석 진행

Date range와 skewness

```
# date range
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')

print("\n=== DATE RANGE ===")
print(df['Date'].min(), df['Date'].max())

=== DATE RANGE ===
2010-02-05 00:00:00 2012-10-26 00:00:00
```

```
# skewness (distributions shape 확인)
print("\n=== SKEWNESS ===")
print(df.skew(numeric_only=True))

=== SKEWNESS ===
Store          0.000000
Weekly_Sales    0.668362
Holiday_Flag    3.373499
Temperature    -0.336768
Fuel_Price     -0.096158
CPI            0.063492
Unemployment    1.188144
dtype: float64
```

변수 간 상관관계 파악

```
# corr matrix (선형 모델 기반 구조 체크)
print("\n=== CORRELATION MATRIX ===")
display(df.corr(numeric_only=True))
```

=== CORRELATION MATRIX ===

	Store	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price	CPI	Unemployment
Store	1.000000e+00	-0.335332	-4.386841e-16	-0.022659	0.060023	-0.209492	0.223531
Weekly_Sales	-3.353320e-01	1.000000	3.689097e-02	-0.063810	0.009464	-0.072634	-0.106176
Holiday_Flag	-4.386841e-16	0.036891	1.000000e+00	-0.155091	-0.078347	-0.002162	0.010960
Temperature	-2.265908e-02	-0.063810	-1.550913e-01	1.000000	0.144982	0.176888	0.101158
Fuel_Price	6.002295e-02	0.009464	-7.834652e-02	0.144982	1.000000	-0.170642	-0.034684
CPI	-2.094919e-01	-0.072634	-2.162091e-03	0.176888	-0.170642	1.000000	-0.302020
Unemployment	2.235313e-01	-0.106176	1.096028e-02	0.101158	-0.034684	-0.302020	1.000000

Weekly_Sales의 자기상관

시계열 자료이므로, 자기상관성이 생길 가능성이 매우 높음

Store 별 Weekly_Sales의 Durbin-Watson 값을 파악하였음

```
# durbin-watson statistic
from statsmodels.stats.stattools import durbin_watson

result = []
for s in sorted(df['Store'].unique()):
    tmp = df[df['Store']==s].sort_values('Date')
    dw = durbin_watson(tmp['Weekly_Sales'])
    result.append([s, dw])

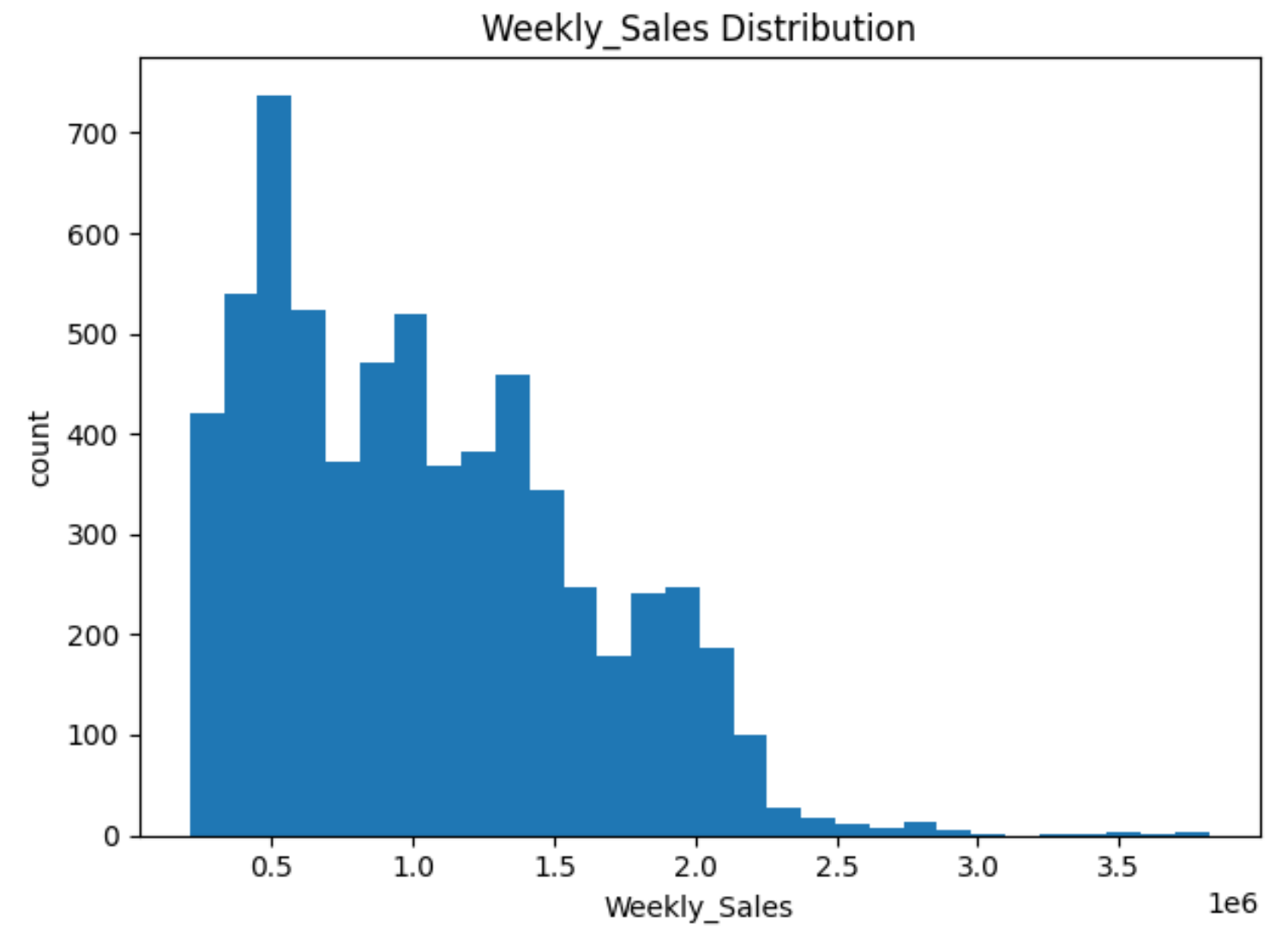
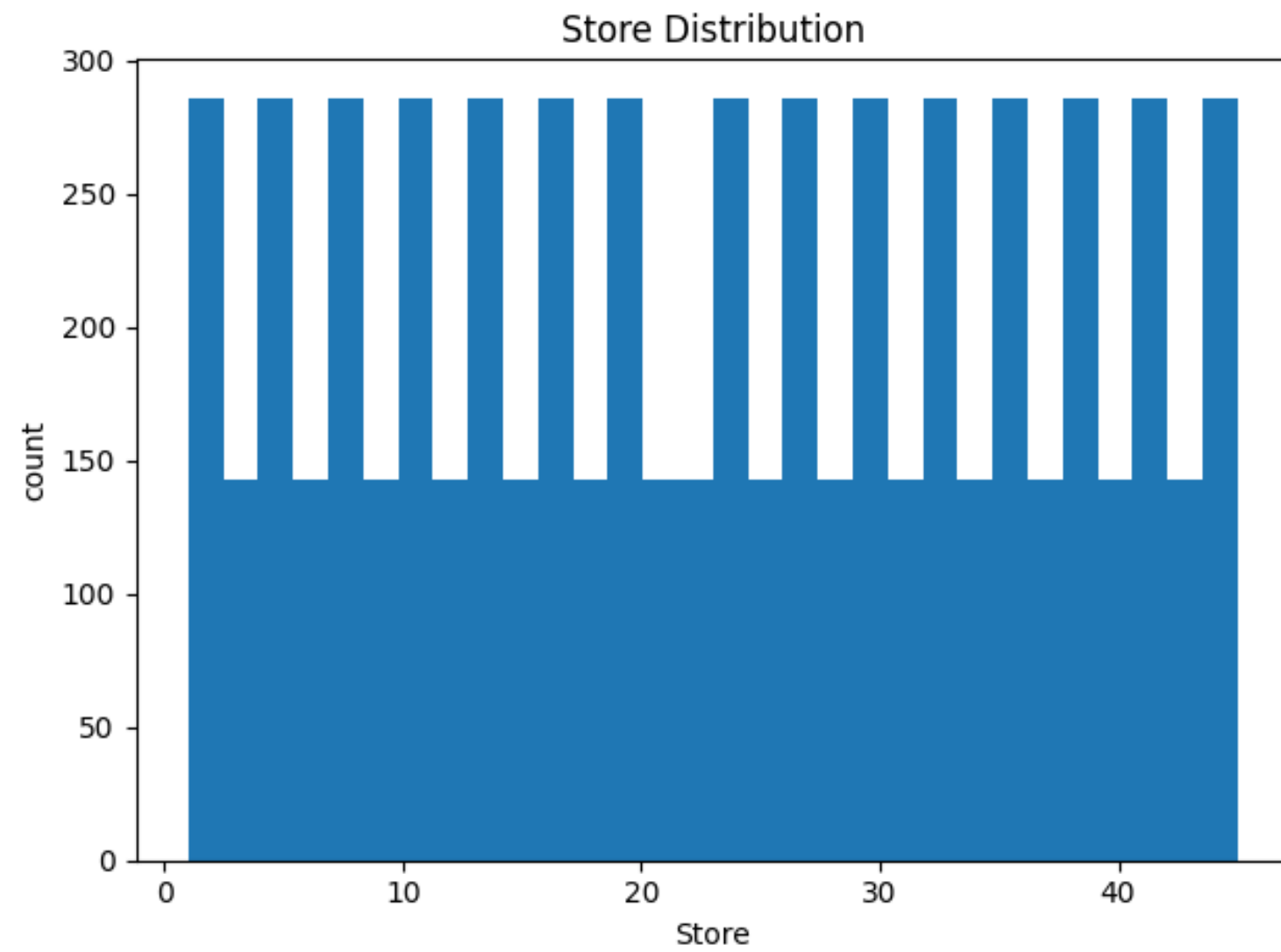
dw_table = pd.DataFrame(result, columns=['Store', 'DW'])

print(dw_table)
```

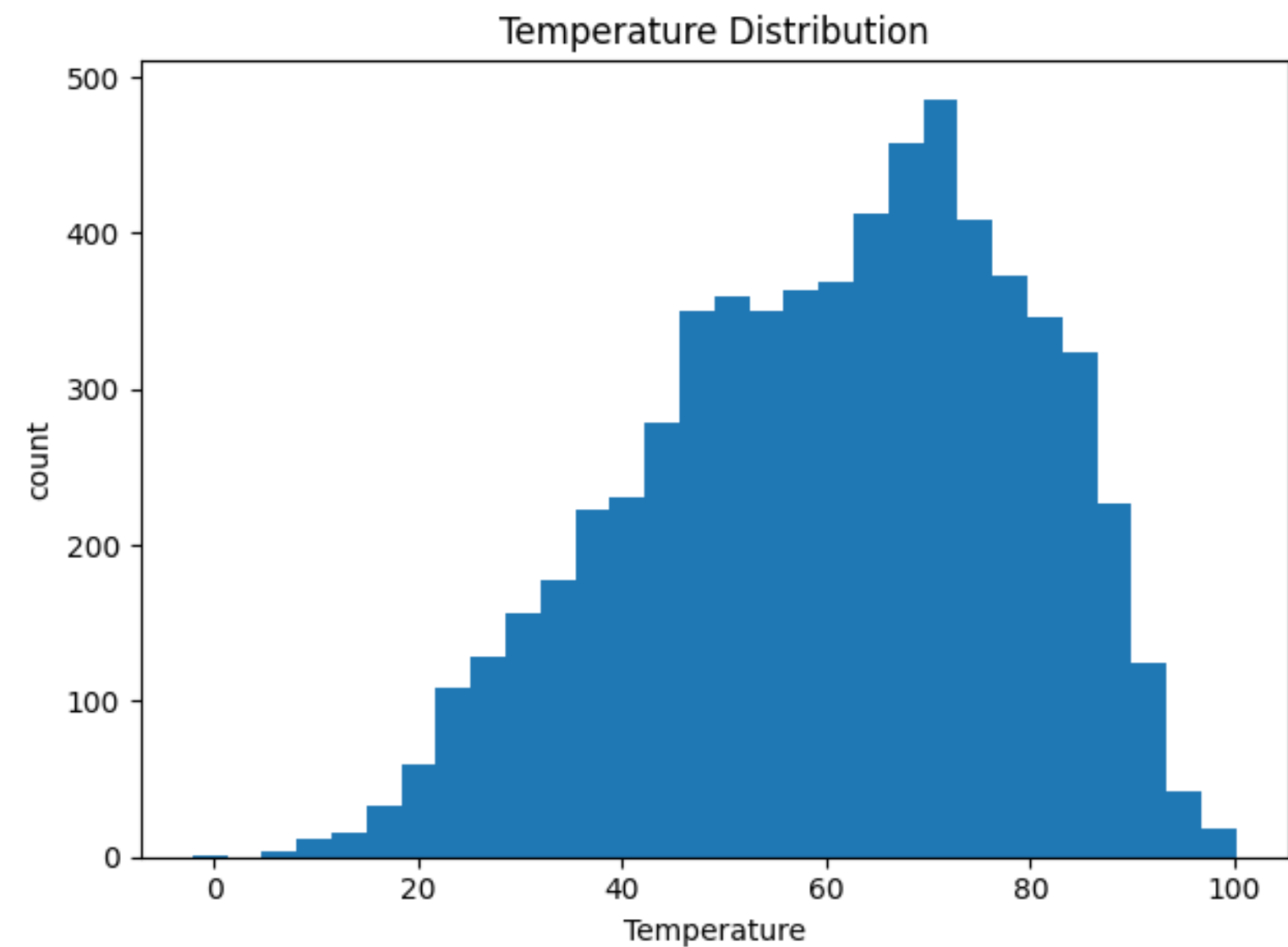
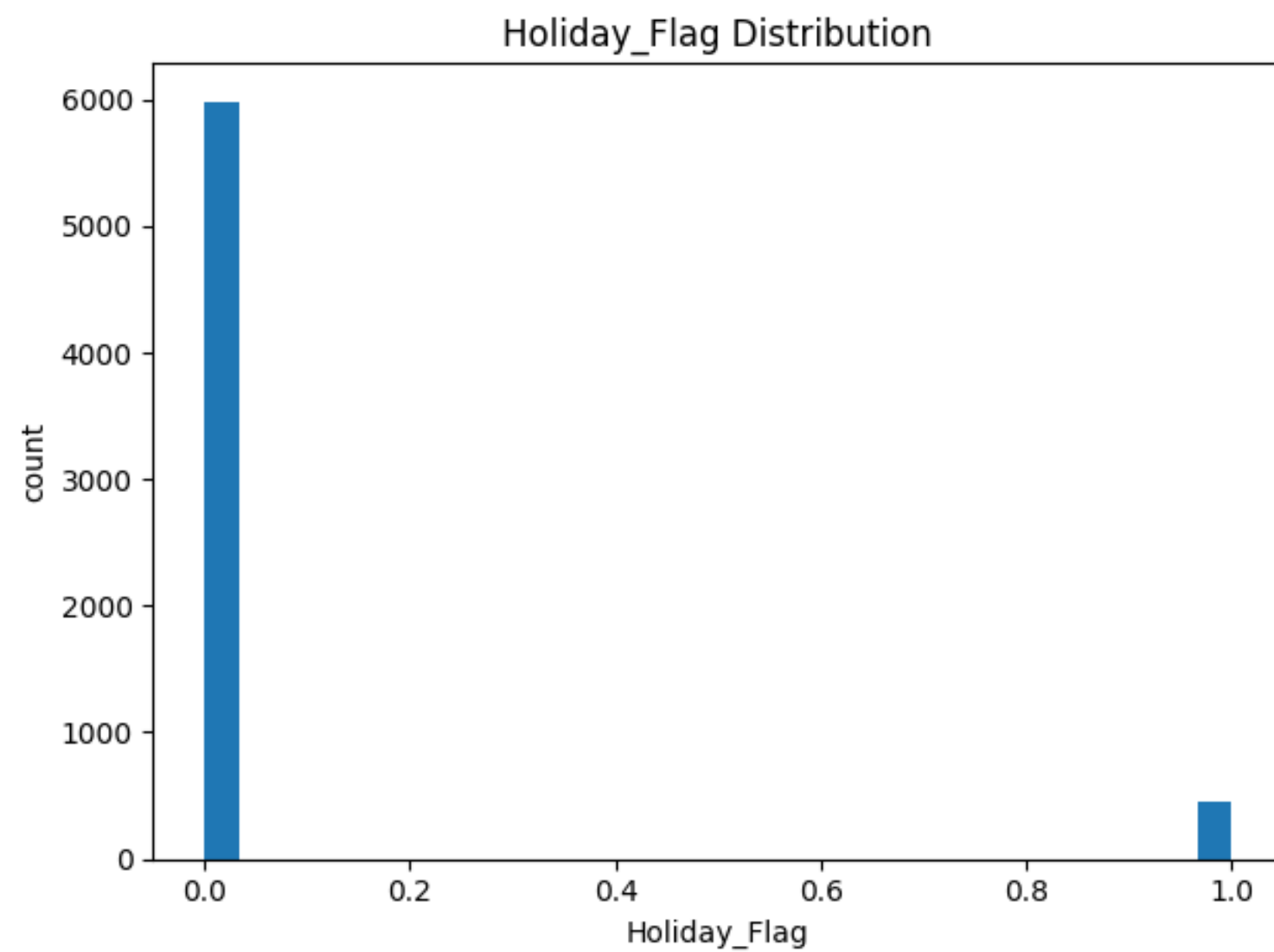
	Store	DW
0	1	0.019153
1	2	0.031364
2	3	0.026015
3	4	0.031679
4	5	0.028865
5	6	0.042943
6	7	0.083574
7	8	0.027000
8	9	0.032814
9	10	0.049428
10	11	0.029887

모든 Store의 DW 값이 0에 매우 가까웠음. 이는 매우 강한 양의 자기상관이 존재함을 시사함

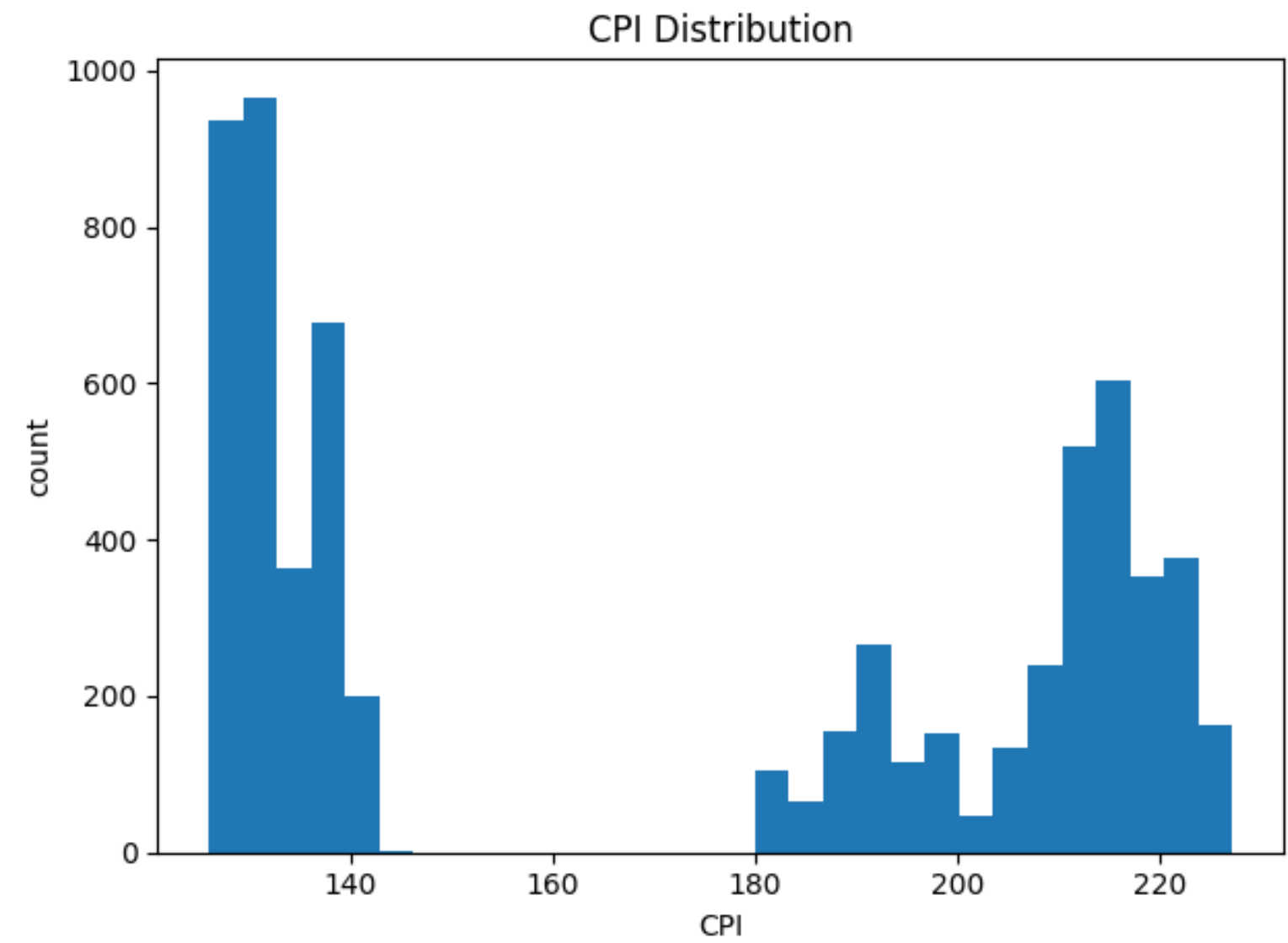
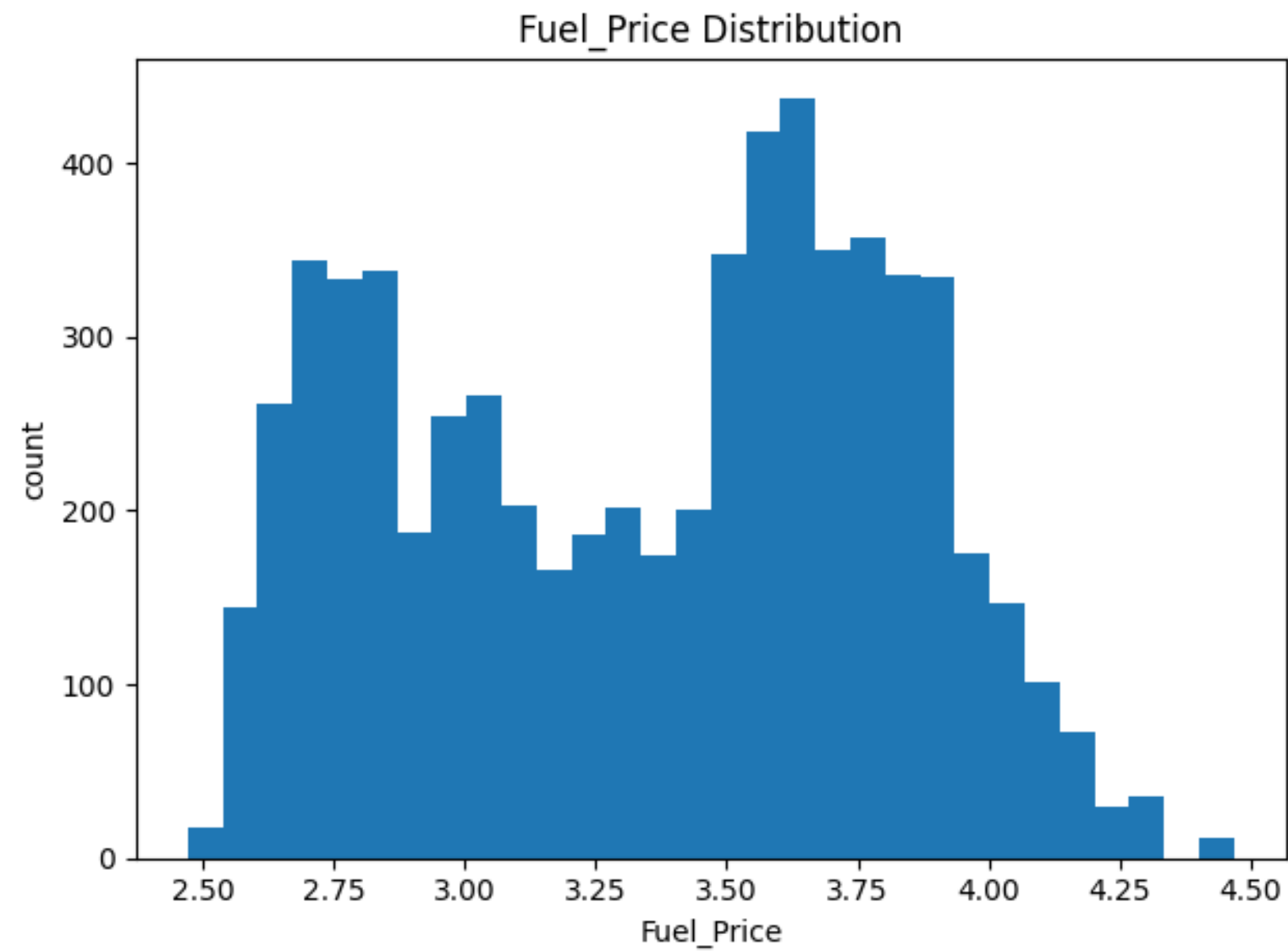
변수 별 히스토그램



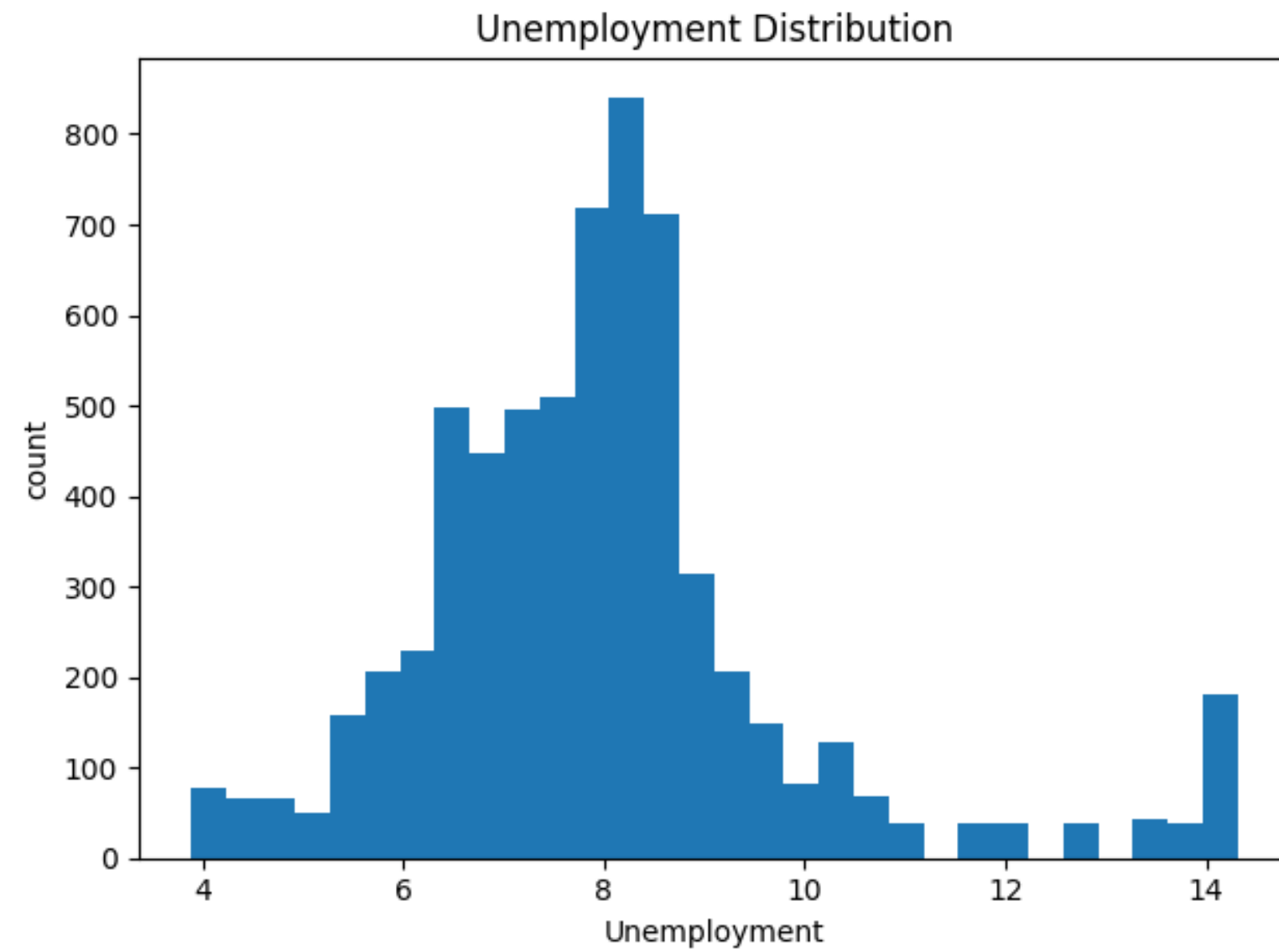
변수 별 히스토그램



변수 별 히스토그램



변수 별 히스토그램



날짜 변환 파생변수 생성

Year(년), Month(월), Week(주), Quarter(분기), DayOfWeek(요일) 변수 생성

```
# 날짜 변환
```

```
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')
```

```
# 날짜 기반 파생변수 생성
```

```
df['Year'] = df['Date'].dt.year
```

```
df['Month'] = df['Date'].dt.month
```

```
df['Week'] = df['Date'].dt.isocalendar().week.astype(int)
```

```
df['Quarter'] = df['Date'].dt.quarter
```

```
df['DayOfWeek'] = df['Date'].dt.dayofweek
```

log 변환

target 변수인 log_Weekly_Sales에 log를 취한 변수인 log_weekly_Sales 변수 생성

log를 취하여 선형회귀 안정화 가능

```
# target 로그 변환 (선형회귀 안정화 목적)  
df['log_Weekly_Sales'] = np.log1p(df['Weekly_Sales'])
```

선형 회귀분석을 위한 전처리 과정에서만 log 변환을 적용하였음(비선형 회귀분석에는 사용X)

lag feature 생성

바로 이전 주의 log_Weekly_Sales를 lag1, 작년 같은 주의 log_Weekly_Sales를 lag52로 생성
데이터가 가지고 있는 연도는 2010, 2011, 2012년도인데, 작년 같은 주의 log_Weekly_Sales
값이 존재해야하므로, 2010년인 데이터는 제거됨

```
# lag feature 생성 (dummy 이전에 해야함)
```

```
df = df.sort_values(['Store', 'Date'])
```

```
# 바로 직전 주 lag (1주 전)
```

```
df['lag1_Weekly_Sales'] = df.groupby('Store')['Weekly_Sales'].shift(1)
```

```
df['lag1_log_Weekly_Sales'] = df.groupby('Store')['log_Weekly_Sales'].shift(1)
```

```
# 작년 동일 주 lag (52주 전)
```

```
df['lag52_Weekly_Sales'] = df.groupby('Store')['Weekly_Sales'].shift(52)
```

```
df['lag52_log_Weekly_Sales'] = df.groupby('Store')['log_Weekly_Sales'].shift(52)
```

```
# lag 생성으로 인한 NaN 행 제거
```

```
df = df.dropna().reset_index(drop=True)
```


test/train 분리

test data는 2011년, train data는 2012년인 data를 이용하였음
또, store 고정효과는 선형 회귀분석 전처리에서만 사용

```
# store 고정효과 (dummy coding) - Linear Regression 에서만 사용
df = pd.get_dummies(df, columns=['Store'], drop_first=True)
```

```
# 시계열 기준 train/test 분리 (2012 기준)
cut_date = pd.to_datetime('2012-01-01')
train_df = df[df['Date'] < cut_date].copy()
test_df = df[df['Date'] >= cut_date].copy()
```

```
# 숫자형 feature scaling (train에만 fit)
scale_cols = ['Temperature', 'Fuel_Price', 'CPI', 'Unemployment',
              'lag1_Weekly_Sales', 'lag1_log_Weekly_Sales',
              'lag52_Weekly_Sales', 'lag52_log_Weekly_Sales']
```

```
scaler = StandardScaler()
scaler.fit(train_df[scale_cols])
```

```
train_df[scale_cols] = scaler.transform(train_df[scale_cols])
test_df[scale_cols] = scaler.transform(test_df[scale_cols])
```

```
print(train_df.shape, test_df.shape)
```

```
(2160, 61) (1935, 61)
```

표준화 과정 또한 선형 회귀분석 전처리에만 사용

train data의 행/열 개수: 2160/61

test data의 행/열 개수: 1935/61

데이터셋이 2012년 10월까지의 자료만을 포함하므로 test data의
행의 개수가 train data의 행의 개수보다 적음

Columns(Linear Regression)

- Date: 날짜
- Weekly_Sales: 주간 판매량
- Holiday_Flag: 휴일이 있는 주
휴일이 있는 주면 1, 아니면 0
- Temperature: 기온(화씨)
- Fuel_Price: 유가
- CPI: 소비자 물가지수
- Unemployment: 실업률
- Year: 년
- Month: 월
- Week: 주
- Quarter: 분기
- DayOfWeek: 요일(전부 4(수요일))
- log_Weekly_Sales: 주간 판매량의 로그 값
- lag1_Weekly_Sales: 직전 주의 주간 판매량
- log_lag1_Weekly_Sales: 전 주의 판매량의 로그 값
- lag52_Weekly_Sales: 작년 동일 주의 주간 판매량
- log_lag52_Weekly_Sales:
작년 동일 주의 주간 판매량의 로그 값
- Store_2, ... Store_45: Store_i는 Store 번호가
i면 TRUE 아니면 FALSE

Columns(Nonlinear Regression)

- Date: 날짜
- Weekly_Sales: 주간 판매량
- Holiday_Flag: 휴일이 있는 주
휴일이 있는 주면 1, 아니면 0
- Temperature: 기온(화씨)
- Fuel_Price: 유가
- CPI: 소비자 물가지수
- Unemployment: 실업률
- Year: 년
- Month: 월
- Week: 주
- Quarter: 분기
- DayOfWeek: 요일(전부 4(수요일))

앞서 다룬대로 비선형 회귀분석을 위한 전처리에서는 log 변환 및 store 고정효과는 진행하지 않았으므로, log 변수, store_i 변수는 없음

3. 선형 회귀분석

사용 모델

1. OLS

2. Ridge

3. Lasso

OLS, Ridge, Lasso

OLS : 변수와 목표값 사이의 선형 관계를 가장 기본적인 방식으로 추정하는 회귀 모델, 오차 제곱합이 최소가 되도록 계수를 계산함

→ 각 변수의 영향 방향과 유의성을 해석하기 쉬움, 모델의 기본적 구조를 파악하기 위한 기준 모델

Ridge : 계수의 크기를 줄이는(L2 패널티) 선형 회귀, 과도하게 큰 계수를 억제해 모델을 안정화함

→ 다중공선성(변수 간 상관) 문제 완화, 과적합 방지

Lasso : 계수에 L1 패널티를 적용해 일부 계수를 정확히 0으로 만드는 회귀 모델

→ 중요하지 않은 변수 제거해 해석성을 높임, 모델을 더 단순하게 만들고 일반화 성능 향상

OLS

[변수 설정 및 데이터 전처리]

- 독립변수(Features) : Temperature, Fuel_Price, CPI, Unemployment, lag1_log_Weekly_Sales, lag52_log_Weekly_Sales
- 종속변수(Target) : log_Weekly_Sales

학습(Train) 데이터

```
X_train = train_data[features]  
y_train = train_data[target]
```

검증(Test) 데이터

```
X_test = test_data[features]  
y_test = test_data[target]
```

OLS 모델은 절편(intercept)을 자동으로 추가하지 않으므로, 수동으로 추가합니다.

```
X_train_const = sm.add_constant(X_train)  
X_test_const = sm.add_constant(X_test)
```

- 학습(Train), 검증(Test) 데이터 분리
(X에는 독립변수, y에는 종속변수 저장)
- OLS 모델용 상수항 추가

OLS

[OLS 모델 학습 및 결과 요약]

=====						
OLS Regression Results						
=====						
OLS Regression Results						
=====						
Dep. Variable:	log_Weekly_Sales	R-squared:	0.985			
Model:	OLS	Adj. R-squared:	0.985			
Method:	Least Squares	F-statistic:	2.349e+04			
Date:	Mon, 10 Nov 2025	Prob (F-statistic):	0.00			
Time:	15:15:14	Log-Likelihood:	2589.1			
No. Observations:	2160	AIC:	-5164.			
Df Residuals:	2153	BIC:	-5125.			
Df Model:	6					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]
=====						
const	13.7087	0.002	8716.346	0.000	13.706	13.712
Temperature	-7.38e-05	0.002	-0.038	0.970	-0.004	0.004
Fuel_Price	-0.0070	0.002	-3.486	0.000	-0.011	-0.003
CPI	-0.0007	0.002	-0.337	0.736	-0.004	0.003
Unemployment	-0.0031	0.002	-1.865	0.062	-0.006	0.000
lag1_log_Weekly_Sales	0.2015	0.006	34.858	0.000	0.190	0.213
lag52_log_Weekly_Sales	0.3937	0.006	68.063	0.000	0.382	0.405
=====						
Omnibus:	596.345	Durbin-Watson:	1.492			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	4746.280			
Skew:	-1.075	Prob(JB):	0.00			
Kurtosis:	9.937	Cond. No.	7.33			
=====						

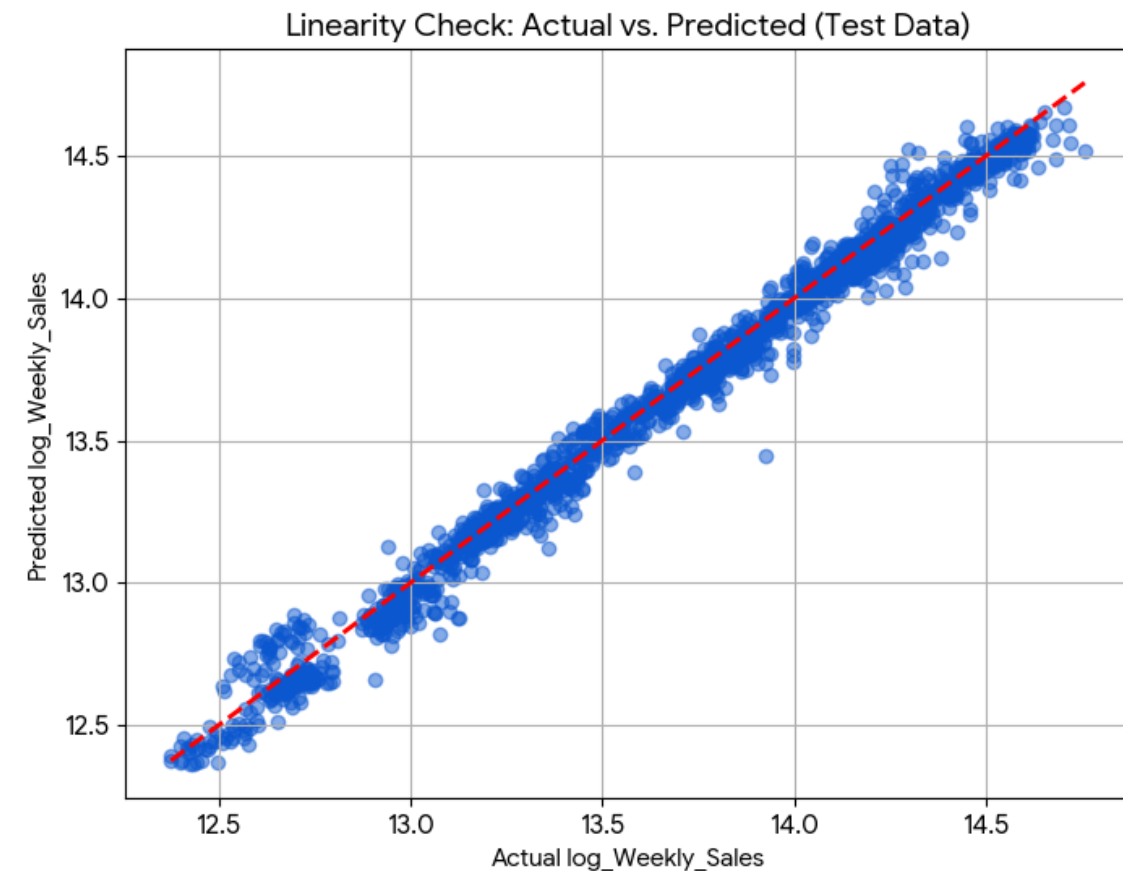
OLS 모델은 R^2 0.985로 높은 설명력을 보였으나
Temperature, CPI 등 일부 변수는
통계적으로 유의하지 않았고
lag 변수(lag1, lag52)가 매출 예측에
가장 큰 영향을 미침

OLS

[선형 회귀분석 기본 가정 검증]

1. 선형성 검증

```
# 1. 선형성 (Linearity) 확인
# - 예측값(test)과 실제값(test)을 비교하는 Scatter plot
print("\n--- 1. 선형성 검증 (Actual vs. Predicted) ---")
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predictions_test, alpha=0.5)
# 추세선 (y=x)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Linearity Check: Actual vs. Predicted (Test Data)')
plt.xlabel('Actual log_Weekly_Sales')
plt.ylabel('Predicted log_Weekly_Sales')
plt.grid(True)
plt.savefig("linearity_check.png")
plt.close() # Close plot to prevent showing in output
print(" 'linearity_check.png' (선형성 검증) 플롯이 생성되었습니다.")
```



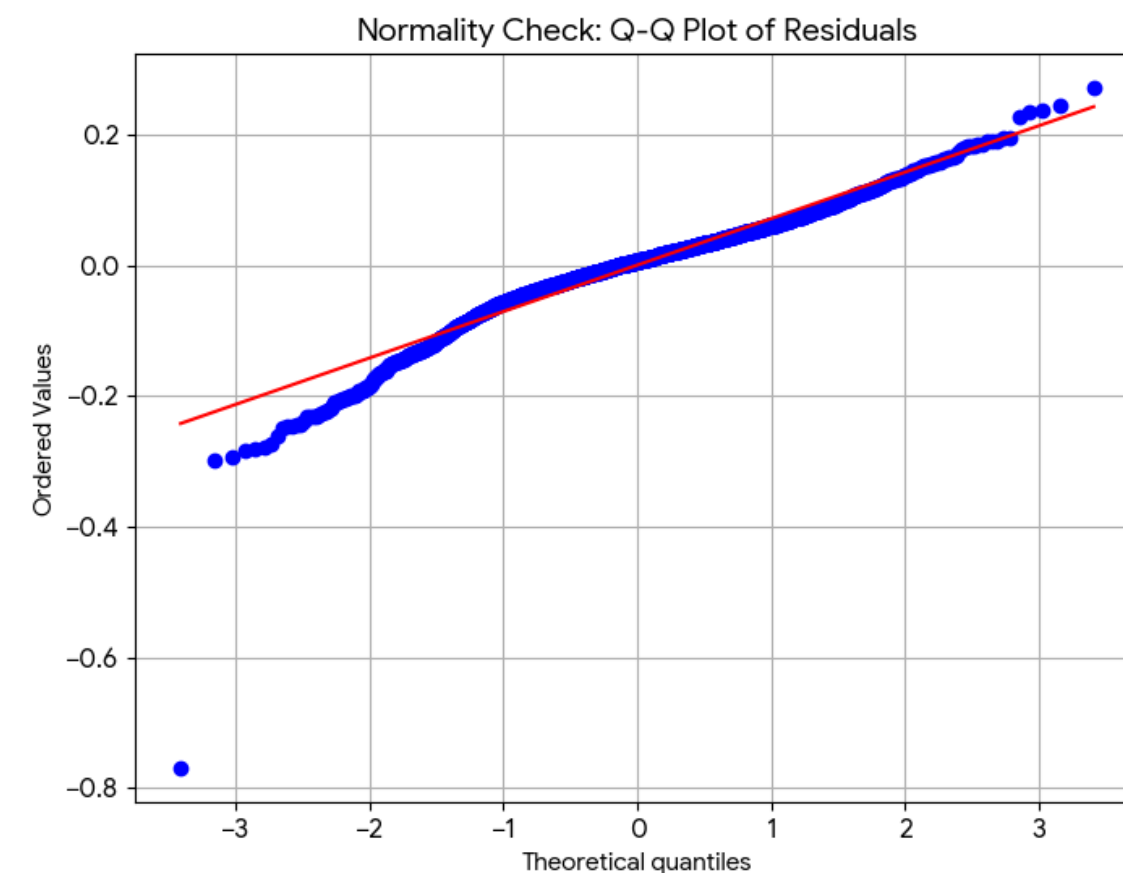
그래프 확인 결과 기준선에 잘 모여있으므로 강한 선형 관계 증명

OLS

[선형 회귀분석 기본 가정 검증]

2. 잔차 정규성 확인 플롯 생성

```
# 2. 잔차의 정규성 (Normality of Residuals) 확인
# - Q-Q Plot
print("\n--- 2. 잔차의 정규성 검증 (Q-Q Plot) ---")
plt.figure(figsize=(8, 6))
stats.probplot(residuals, dist="norm", plot=plt)
plt.title('Normality Check: Q-Q Plot of Residuals')
plt.grid(True)
plt.savefig("qq_plot.png")
plt.close() # Close plot
print(" 'qq_plot.png' (Q-Q 플롯)이 생성되었습니다.")
```



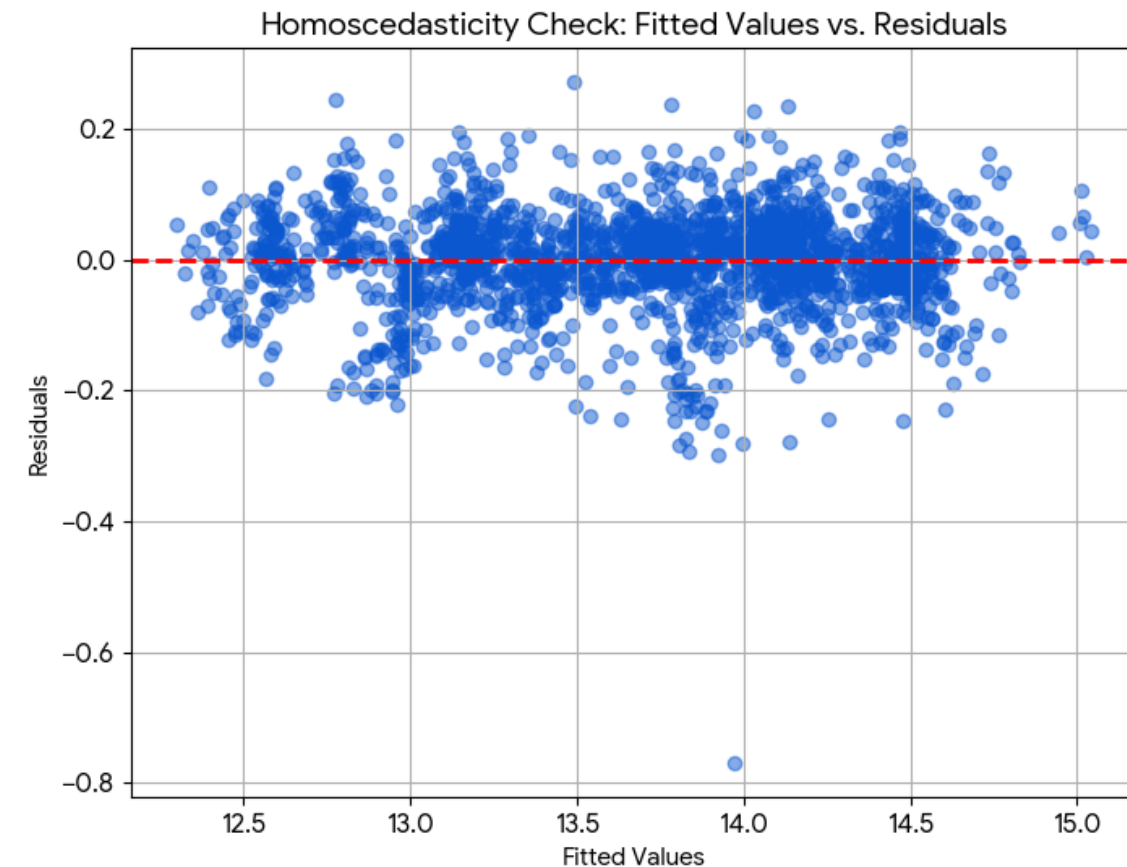
그래프 확인 결과 그래프의 양쪽 끝단에서 기준선을 크게 벗어나 있으므로, 평균에서 멀리 떨어진 극단적인 오차 값들이 예상보다 많음.

OLS

[선형 회귀분석 기본 가정 검증]

3. 잔차의 등분산성 확인 플롯 생성

```
# 3. 잔차의 등분산성 (Homoscedasticity) 확인
# - 예측값(fitted values)과 잔차(residuals)의 Scatter plot
print("\n--- 3. 잔차의 등분산성 검증 (Residual Plot) ---")
plt.figure(figsize=(8, 6))
plt.scatter(fitted_values, residuals, alpha=0.5)
# 0 기준선
plt.axhline(y=0, color='r', linestyle='--', lw=2)
plt.title('Homoscedasticity Check: Fitted Values vs. Residuals')
plt.xlabel('Fitted Values')
plt.ylabel('Residuals')
plt.grid(True)
plt.savefig("residual_plot.png")
plt.close() # Close plot
print(" 'residual_plot.png' (잔차 플롯)이 생성되었습니다.")
```



잔차의 상하 폭이 넓고, 양쪽 끝으로 갈수록 폭이 좁아지므로
즉, 오차의 분산이 일정하지 않으므로 이분산성을 의미함.

OLS

[선형 회귀분석 기본 가정 검증]

4. 다중공선성 확인

```
# 4. 다중공선성 (Multicollinearity) 확인
# - VIF (Variance Inflation Factor) 계산
# - VIF는 독립변수들(X_train)만으로 계산합니다. (상수항 제외)
print("\n--- 4. 다중공선성 검증 (VIF) ---")
vif_data = pd.DataFrame()
vif_data["feature"] = X_train.columns
vif_data["VIF"] = [variance_inflation_factor(X_train.values, i) for i in range(len(X_train.columns))]

print(vif_data)
print("\nVIF 해석:")
print("- VIF가 10 이상이면 다중공선성이 높다고 판단합니다 (일부 기준에서는 5).")
print("- VIF가 1에 가까울수록 다중공선성 문제가 적습니다.")
print("\n--- 모든 분석 완료 ---")
```

--- 4. 다중공선성 검증 (VIF) ---

	feature	VIF
0	Temperature	1.516491
1	Fuel_Price	1.643461
2	CPI	1.506441
3	Unemployment	1.134258
4	lag1_log_Weekly_Sales	13.513324
5	lag52_log_Weekly_Sales	13.523859

VIF 해석:

- VIF가 10 이상이면 다중공선성이 높다고 판단합니다 (일부 기준에서는 5).
- VIF가 1에 가까울수록 다중공선성 문제가 적습니다.

다중공선성 문제가 존재함을 알 수 있음

lag1 변수와 lag52 변수가 서로 강한 상관관계를 가지고 있음을 의미함

(ex) 작년 매출이 높으면 올해 지난주 매출도 높을 경향)

이로 인해 각 변수의 영향력 값이 부정확하게 추정될 수 있음

Ridge & Lasso

[데이터 스케일링]

```
# 3. 데이터 스케일링 (Ridge/Lasso의 필수 전처리)
# -----
# Ridge와 Lasso는 규제(Regularization)를 사용하므로 변수들의 스케일에 민감합니다.
# StandardScaler를 사용하여 모든 변수를 표준화합니다 (평균 0, 분산 1)

scaler = StandardScaler()
# 훈련 데이터로 스케일러를 '학습(fit)'시키고 '변환(transform)'합니다.
X_train_scaled = scaler.fit_transform(X_train)
# 테스트 데이터는 훈련 데이터의 스케일 기준으로 '변환(transform)'만 합니다.
X_test_scaled = scaler.transform(X_test)
```

Ridge/Lasso 회귀는 변수의 스케일 차이에 민감하므로,
StandardScaler를 사용해 모든 변수를 평균 0, 분산 1로 표준화하였다

Ridge & Lasso

[OLS (Scaled) - 비교 기준 모델]

```
# 4. OLS (Scaled) - 비교 기준 모델
# -----
# Ridge/Lasso 계수와 공정한 비교를 위해 스케일링된 데이터로 OLS를 실행합니다.
X_train_const_scaled = sm.add_constant(X_train_scaled)
X_test_const_scaled = sm.add_constant(X_test_scaled)

ols_model_scaled = sm.OLS(y_train, X_train_const_scaled).fit()
# OLS의 테스트 R-squared 계산
ols_pred_test = ols_model_scaled.predict(X_test_const_scaled)
ols_r2_test = r2_score(y_test, ols_pred_test)

print(f"OLS (Scaled) Test R-squared: {ols_r2_test:.4f}")
# 계수 (상수항 제외)
ols_coefs = ols_model_scaled.params[1:]
```

OLS (Scaled) Test R-squared: 0.9879

Ridge & Lasso

[Ridge 회귀분석]

```
# 5. Ridge 회귀분석 (RidgeCV)
# -----
# RidgeCV는 교차 검증(Cross-Validation)을 통해 최적의 alpha 값을 자동으로 찾아줍니다.
# 테스트할 alpha 값들의 범위 설정
alphas_ridge = np.logspace(-3, 3, 100) # 10^-3 에서 10^3 까지 100개

ridge_cv = RidgeCV(alphas=alphas_ridge, cv=5)
ridge_cv.fit(X_train_scaled, y_train)

# 최적의 alpha
print(f"\\n--- Ridge Regression (RidgeCV) ---")
print(f"Best Alpha: {ridge_cv.alpha_:.4f}")

# 테스트 R-squared
ridge_r2_test = ridge_cv.score(X_test_scaled, y_test)
print(f"Ridge Test R-squared: {ridge_r2_test:.4f}")

# Ridge 계수
ridge_coefs = ridge_cv.coef_
```

- Best Alpha: 23.1013
- Ridge Test R-squared: 0.9884

OLS에서 나타난 lag 변수 간 높은 다중공선성을 완화
계수의 크기를 안정적으로 조정해 예측 성능을 유지

Ridge & Lasso

[Lasso 회귀분석]

```
# 6. Lasso 회귀분석 (LassoCV)
# -----
# LassoCV도 교차 검증으로 최적의 alpha를 찾습니다.
alphas_lasso = np.logspace(-6, 0, 100) # 10^-6 에서 10^0 까지 100개

lasso_cv = LassoCV(alphas=alphas_lasso, cv=5, max_iter=10000, random_state=42)
lasso_cv.fit(X_train_scaled, y_train)

# 최적의 alpha
print(f"\n--- Lasso Regression (LassoCV) ---")
print(f"Best Alpha: {lasso_cv.alpha_:.4f}")

# 테스트 R-squared
lasso_r2_test = lasso_cv.score(X_test_scaled, y_test)
print(f"Lasso Test R-squared: {lasso_r2_test:.4f}")

# Lasso 계수
lasso_coefs = lasso_cv.coef_
```

- Best Alpha: 0.0022
- Lasso Test R-squared: 0.9880

Temperature와 CPI 계수를 정확히 0으로 만들어
변수 선택을 수행
두 lag 변수를 중심으로 OLS·Ridge와 동일한 수준의
높은 예측력을 유지

Ridge & Lasso

[계수 비교]

```
# 7. 계수 비교
# -----
print("\n" + "="*80)
print("                Coefficient Comparison (Scaled Data)")
print("="*80)

coef_df = pd.DataFrame({
    'feature': features,
    'OLS_Scaled_Coef': ols_coefs.values,
    'Ridge_Coef': ridge_coefs,
    'Lasso_Coef': lasso_coefs
})

print(coef_df)
print("="*80)
```

feature	OLS_Scaled_Coef	Ridge_Coef	Lasso_Coef
Temperature	-0.000074	-0.000744	-0.000000
Fuel_Price	-0.007029	-0.006249	-0.004947
CPI	-0.000650	-0.000663	0.000000
Unemployment	-0.003123	-0.003249	-0.001437
lag1_log_Weekly_Sales	0.201534	0.221121	0.201624
lag52_log_Weekly_Sales	0.393663	0.370746	0.391585

Ridge & Lasso

[가정 검증]

```
# 8. 가정 검증을 위한 값 계산
# -----

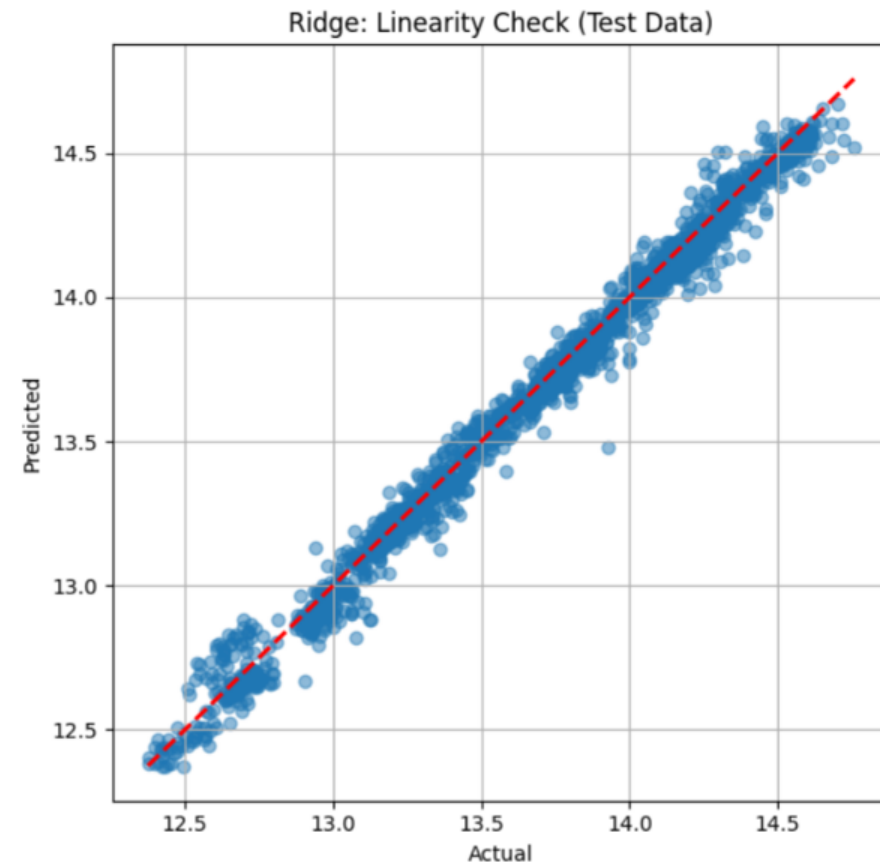
# ----- Ridge 모델 값 -----
# 훈련 데이터에 대한 예측값 (등분산성, 정규성 검증용)
fitted_values_train_ridge = ridge_cv.predict(X_train_scaled)
# 훈련 데이터의 잔차 (등분산성, 정규성 검증용)
residuals_train_ridge = y_train - fitted_values_train_ridge
# 테스트 데이터에 대한 예측값 (선형성 검증용)
predictions_test_ridge = ridge_cv.predict(X_test_scaled)

# ----- Lasso 모델 값 -----
# 훈련 데이터에 대한 예측값
fitted_values_train_lasso = lasso_cv.predict(X_train_scaled)
# 훈련 데이터의 잔차
residuals_train_lasso = y_train - fitted_values_train_lasso
# 테스트 데이터에 대한 예측값
predictions_test_lasso = lasso_cv.predict(X_test_scaled)
```

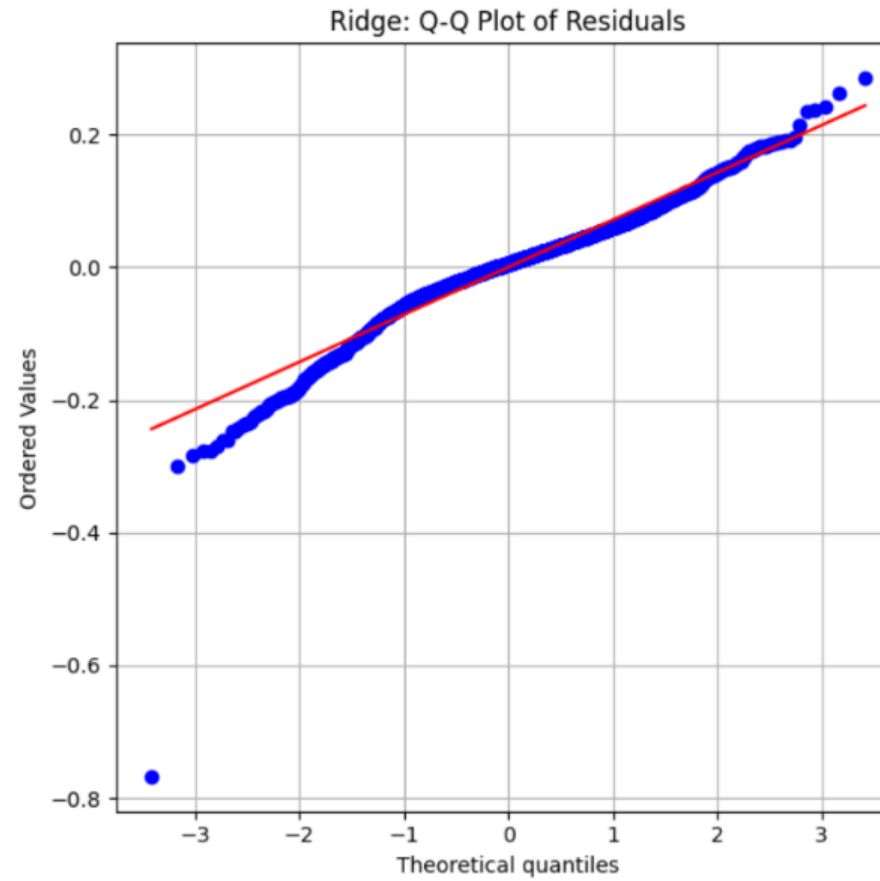
Ridge와 Lasso 가정 검증을 위해
모델 값 계산을 진행

Ridge & Lasso

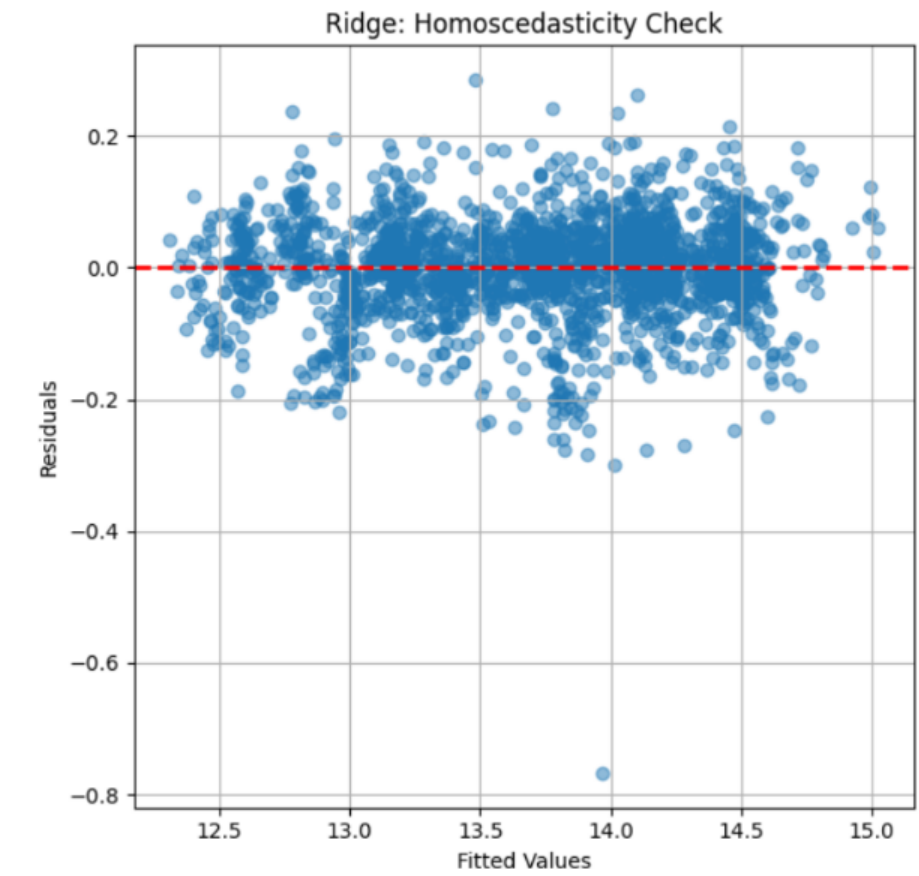
1. Ridge 가정 검증



(1) 선형성 만족



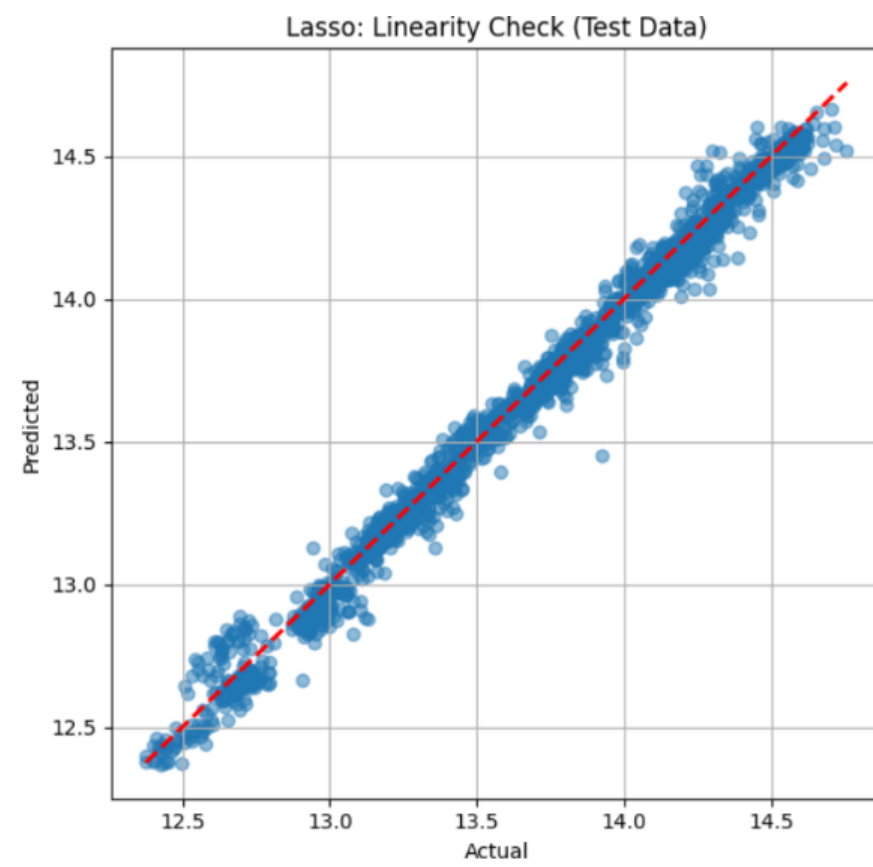
(2) 잔차 정규성 위반



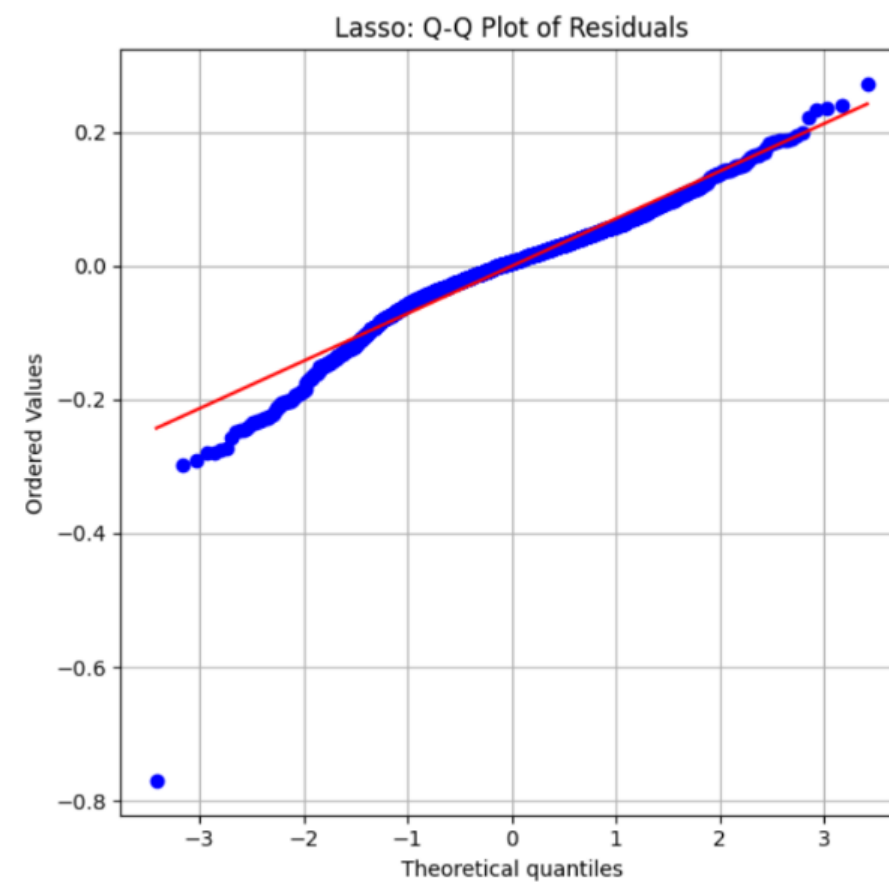
(3) 잔차 등분산성 위반

Ridge & Lasso

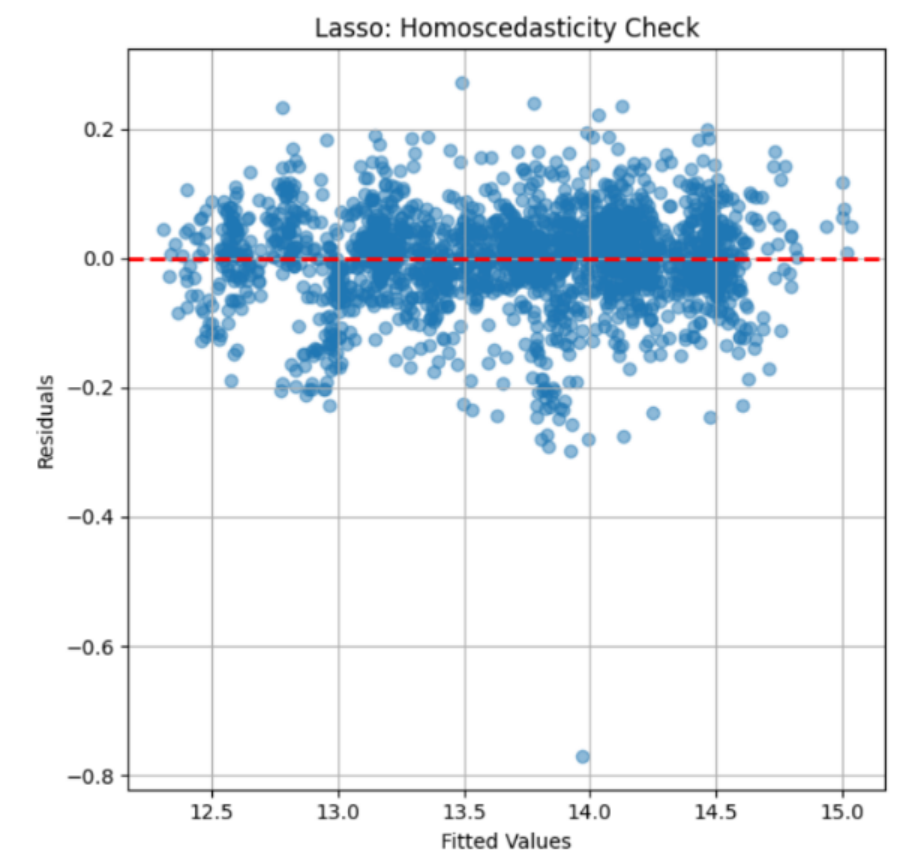
2. Lasso 가정 검증



(1) 선형성 만족



(2) 잔차 정규성 위반



(3) 잔차 등분산성 위반

최종 채택

- 세 모델 모두 예측력은 거의 동일하게 매우 뛰어난 편
- Ridge는 lag1과 lag52의 영향력을 재분배하여 OLS의 불안정한 계수 문제 해결
- Lasso는 lag의 영향력은 유지하면서 OLS에서 유의미하지 않았던 변수 Temperature, CPI를 모델에서 완전히 제거

단순한 예측 목적이면 어느 모델을 써도 무방하나,
모델의 간편성과 핵심 변수 파악만을 목적으로 한다면 **Lasso**가 좋은 선택이라고 판단
(변수를 줄여도 OLS와 동일한 성능을 내기 때문)

4. 비선형 회귀분석

사용 모델

1. RandomForest

2. GradientBoostingRegressor

3. LightGBM

4. XGBoost

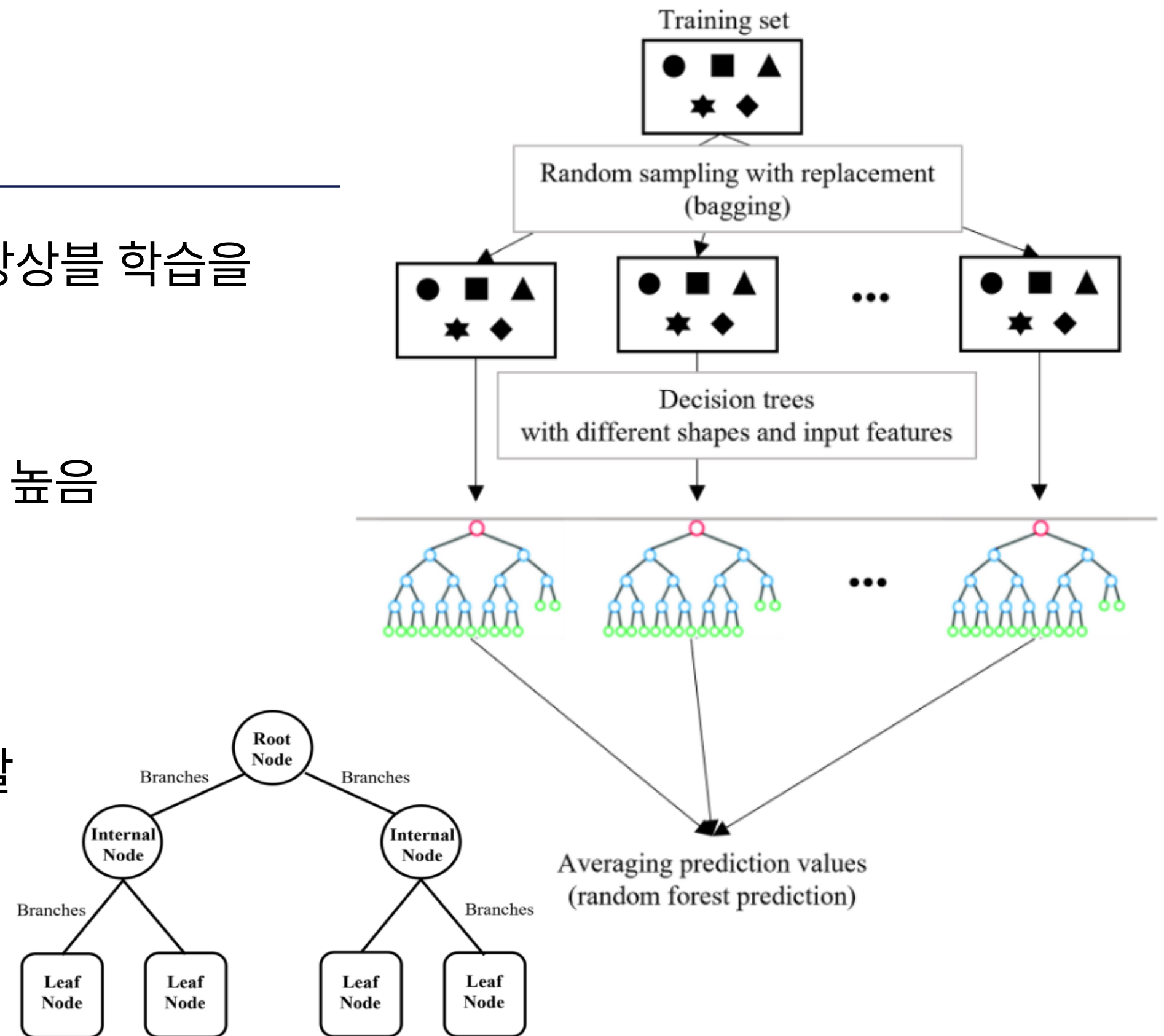
RandomForest

의사결정나무에 분산을 줄이기 위한 배깅 앙상블 학습을 적용하고 의사결정나무들을 조합함

장점: 과적합 해소 및 분산 감소로 정확도가 높음

단점: 계산 비용이 높고 규칙이 많음

의사결정나무: 하나의 특징을 골라
특정 기준 값에 따라 2개의 자식마디로 분할
끝마디는 출력변수 값



RandomForest

[타겟 변수(Weekly_Sales) 자동 탐색]

```
# 2) 타겟(Weekly Sales) 자동 탐색
# =====
def find_target_column(cols):
    prefs = ["Weekly_Sales", "weekly_sales", "WeeklySales", "Weekly_Sale", "sales", "Sales"]
    for p in prefs:
        if p in cols:
            return p
    for c in cols:
        if "weekly" in c.lower() and "sale" in c.lower():
            return c
    return None

y_col = find_target_column(train.columns)
X_train = train.drop(columns=[y_col])
y_train = train[y_col]
X_test = test.drop(columns=[y_col], errors="ignore")
y_test = test[y_col]
```

Weekly Sales 탐색

학습용과 테스트용 데이터의
입력변수(X)와 타겟(y)을 분리

RandomForest

[숫자형 변수와 범주형 변수 분리]

```
# 3) 숫자 / 범주형 분리 + 전처리
# =====
num_cols = [c for c in X_train.columns if pd.api.types.is_numeric_dtype(X_train[c])]
cat_cols = [c for c in X_train.columns if c not in num_cols]
```

숫자형 변수 열과 범주형 변수 열을 추출하여,
숫자형 변수면 num_cols, 범주형 변수면 cat_cols

RandomForest

[원-핫 인코딩 적용]

```
# OneHotEncoder 버전 호환
try:
    ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(handle_unknown="ignore", sparse=False)

preproc = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("ohe", ohe)
        ]), cat_cols)
    ]
)
```

원-핫 인코딩 객체를 생성

숫자형 변수는 결측치를 중위값,
범주형 변수는 결측치를 최빈값으로
채우고 원-핫 인코딩 적용

RandomForest

[모델 생성 및 학습 수행]

```
# =====  
# 4) 랜덤 포레스트 모델 생성  
# =====  
model = Pipeline([  
    ("preproc", preproc),  
    ("rf", RandomForestRegressor(  
        n_estimators=500,  
        random_state=42,  
        n_jobs=-1  
    ))  
)  
  
# =====  
# 5) 학습  
# =====  
model.fit(X_train, y_train)
```

전처리와 랜덤포레스트 회귀로 구성된
전체 파이프라인을 구성.
트리 개수=500, 무작위 시드=42로 하여
병렬 처리.

모델 학습 수행

RandomForest

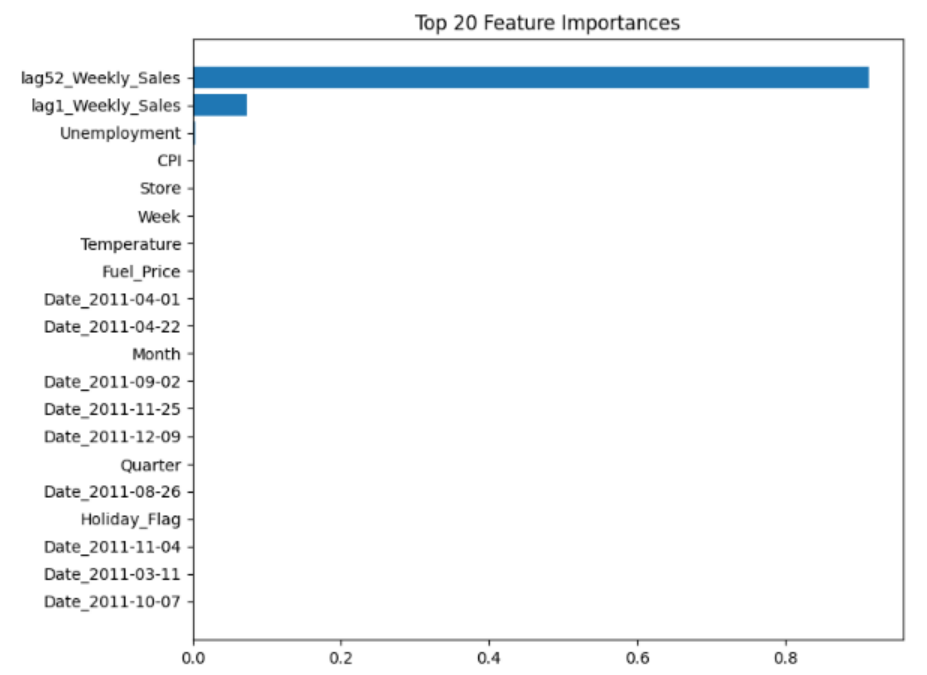
[그래프 및 결과 해석]

Train

R^2 : 0.998432282601149
RMSE : 22808.955392720683
MAE : 14616.640537907393

Test

R^2 : 0.9826645439449336
RMSE : 70639.77646667442
MAE : 45796.26342403109



RMSE 값 자체는 Weekly_Sales가
수백만 원 단위로 구성된 데이터
특성상 크게 나타난 것이며,
실제 매출 규모 대비 RMSE 비율이
약 6% 수준으로 매우 양호한 예측 정확도

Train RMSE 비율(%) : 6.019907388674369

Test RMSE 비율(%) : 6.6259761937541874

GBM, LightGBM, XGBoost

셋 모두 이전 트리의 오류를 줄여나가며 순차적으로 학습하는 부스팅 앙상블 학습 적용

GBM : 기본 알고리즘으로, 순차적으로 속도가 느리고 과적합 방지 기능이 부족

XGBoost: GBM의 단점을 보완하여 정규화 기능과 병렬 처리 등 도입

→ GBM보다 더 견고하고 빠른 모델 구축이 가능

LightGBM: XGBoost의 속도 문제를 개선

Leaf-wise 트리 성장 방식과 Histogram 기반의 분할 기법을 사용

→ 학습 및 예측 시간의 획기적 단축, 대규모 데이터셋에 특히 효율적

GBM

[하이퍼파라미터 탐색]

```
# 모델 및 하이퍼파라미터 탐색
gbr = GradientBoostingRegressor(random_state=42)

pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("model", gbr)
])

param_distributions = {
    "model__n_estimators": randint(100, 400),
    "model__learning_rate": uniform(0.01, 0.2),
    "model__max_depth": randint(2, 6),
    "model__subsample": uniform(0.7, 0.3)
}

search = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_distributions,
    n_iter=12,
    cv=3,
    scoring="neg_root_mean_squared_error",
    verbose=1,
    random_state=42,
    n_jobs=-1
)

print("\n 모델 학습 시작...")
search.fit(X_train, y_train)
print("Best parameters:", search.best_params_)

best_model = search.best_estimator_
```

GridSearchCV로 최적 하이퍼파라미터를 탐색

learning_rate=0.14
max_depth=2
n_estimators=357
subsample=0.92

GBM

[성능 평가]

성능 평가

```
y_pred = best_model.predict(X_test)
```

```
r2 = r2_score(y_test, y_pred)
```

mean_squared_error에서 squared 인자 제거 → 직접 제곱근 계산

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
explained_var = explained_variance_score(y_test, y_pred)
```

```
rmse_ratio = rmse / np.mean(y_test)
```

```
print("\n 모델 평가")
```

```
print(f"R² : {r2:.4f}")
```

```
print(f"RMSE : {rmse:.2f}")
```

```
print(f"MAE : {mae:.2f}")
```

```
print(f"Explained Variance: {explained_var:.4f}")
```

```
print(f"RMSE / Mean Sales: {rmse_ratio:.4f}")
```

모델 평가

R²: 0.9764

RMSE : 82365.60

MAE : 57947.33

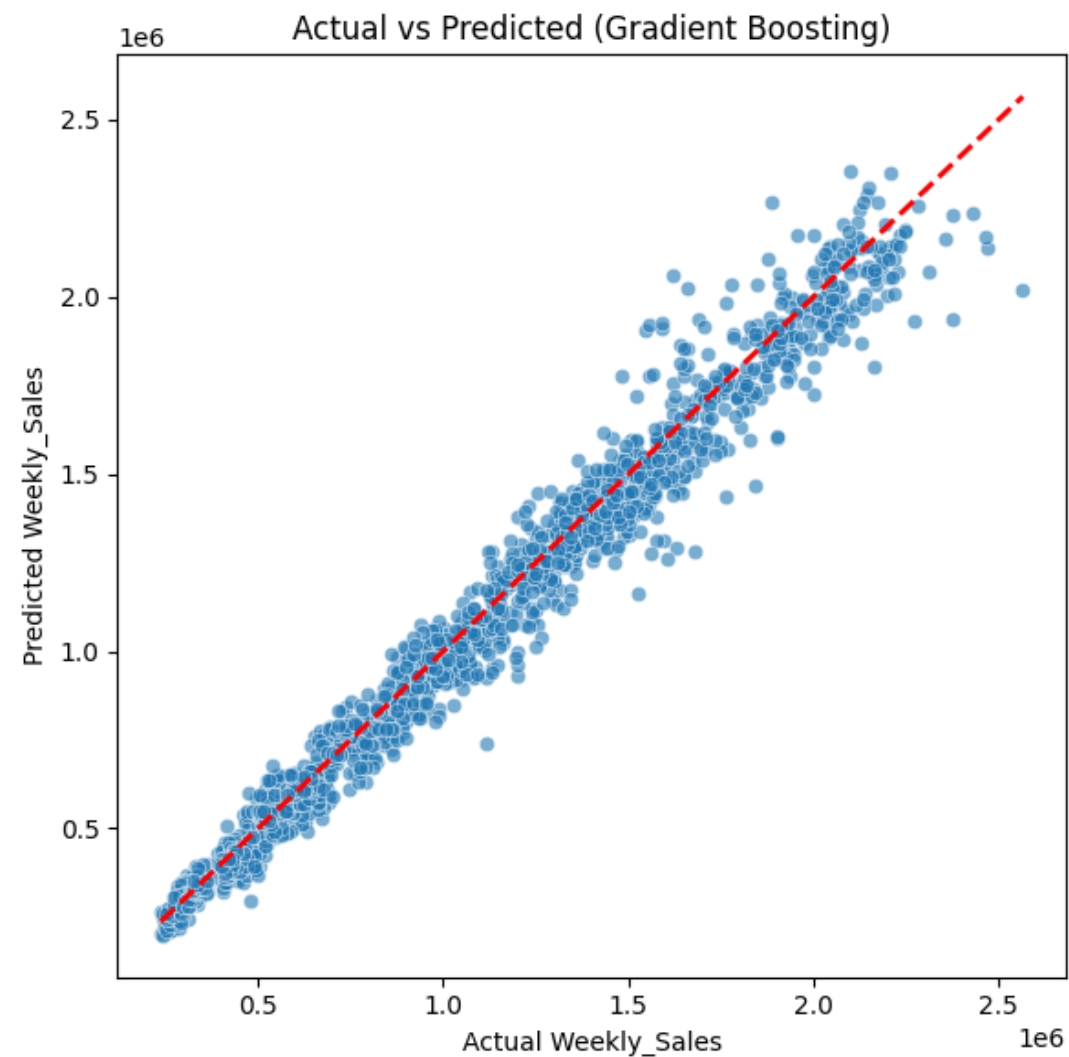
Explained Variance: 0.9777

RMSE / Mean Sales: 0.0797

높은 R²와 낮은 상대 RMSE로 전반적으로 우수한 예측력을 확인 가능

GBM

[실제값 vs 예측값]



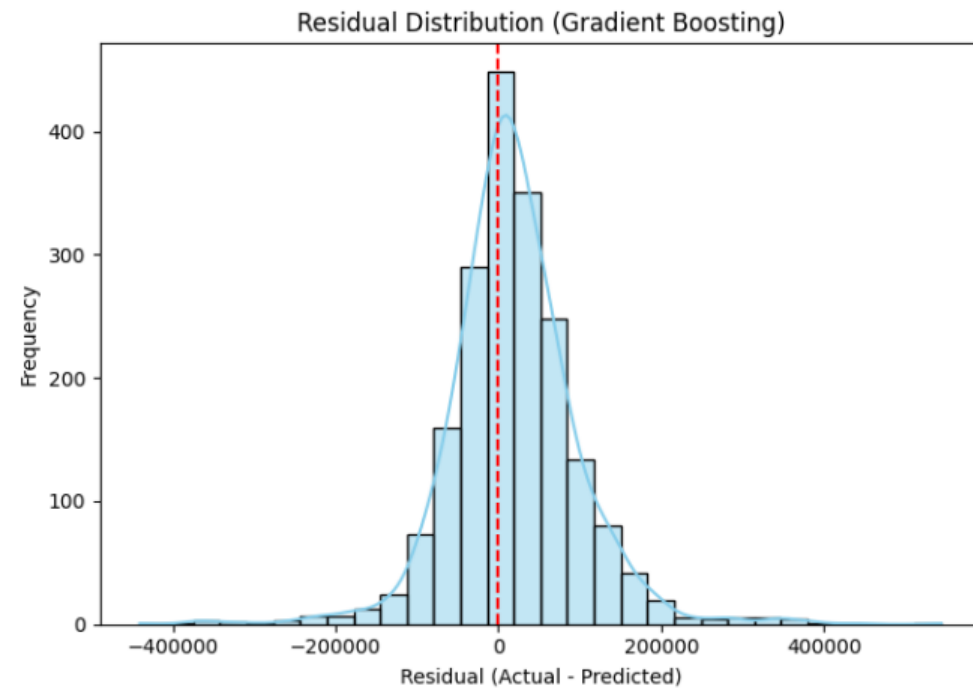
```
# 예측 수행
y_pred = best_model.predict(X_test)
residuals = y_test - y_pred

# 실제값 vs 예측값
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Weekly_Sales")
plt.ylabel("Predicted Weekly_Sales")
plt.title("Actual vs Predicted (Gradient Boosting)")
plt.tight_layout()
plt.show()
```

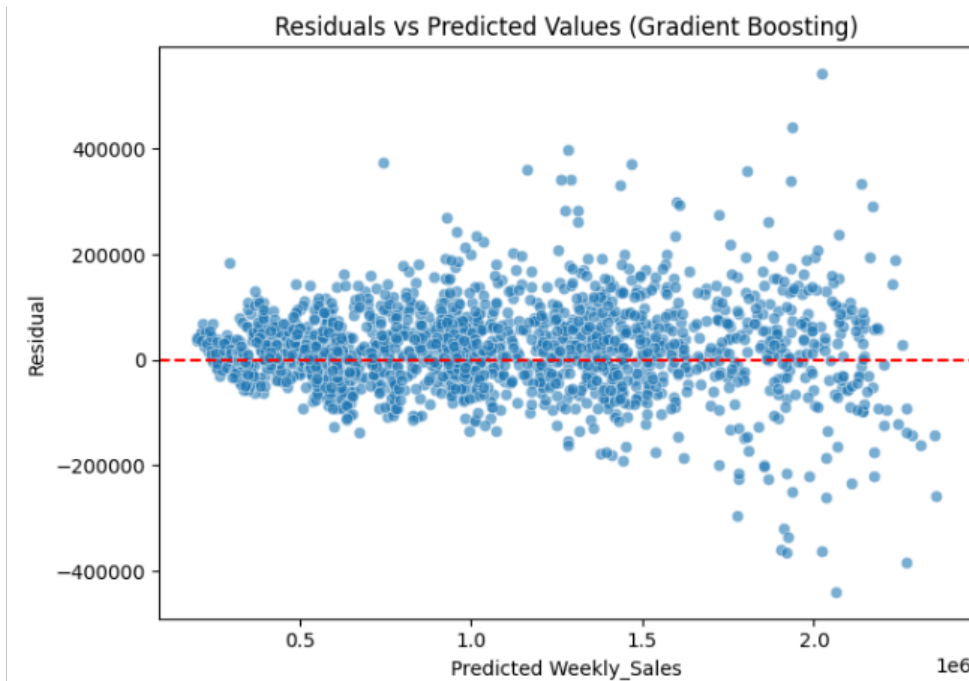
전 구간에서 실제 매출과 예측 매출이 선형적으로 잘 일치하며 안정적인 예측력을 확인

GBM

[Residual Summary Statistics]



잔차 분포 히스토그램



잔차 vs 예측값

count 1935.000000
mean 18762.215464
std 80220.931491
min -440406.495832
25% -24506.888485
50% 14997.487215
75% 59008.622662
max 543940.815504
Name: Weekly_Sales, dtype: float64

RMSE: 82365.60
MAE: 57947.33
Mean Residual: 18762.22

GBM

[과적합 점검 결과]

```
# 과적합 점검
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

# Train 데이터 예측
y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)

# RMSE 직접 계산
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

metrics = {}
for name, y_true, y_pred in [("Train", y_train, y_train_pred), ("Test", y_test, y_test_pred)]:
    r2 = r2_score(y_true, y_pred)
    rmse_val = rmse(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    ratio = rmse_val / np.mean(y_true)
    metrics[name] = {"R2": r2, "RMSE": rmse_val, "MAE": mae, "RMSE/Mean": ratio}
```

Train

R² : 0.9949
RMSE : 41202.64
MAE : 28826.01
RMSE/Mean = 0.0390

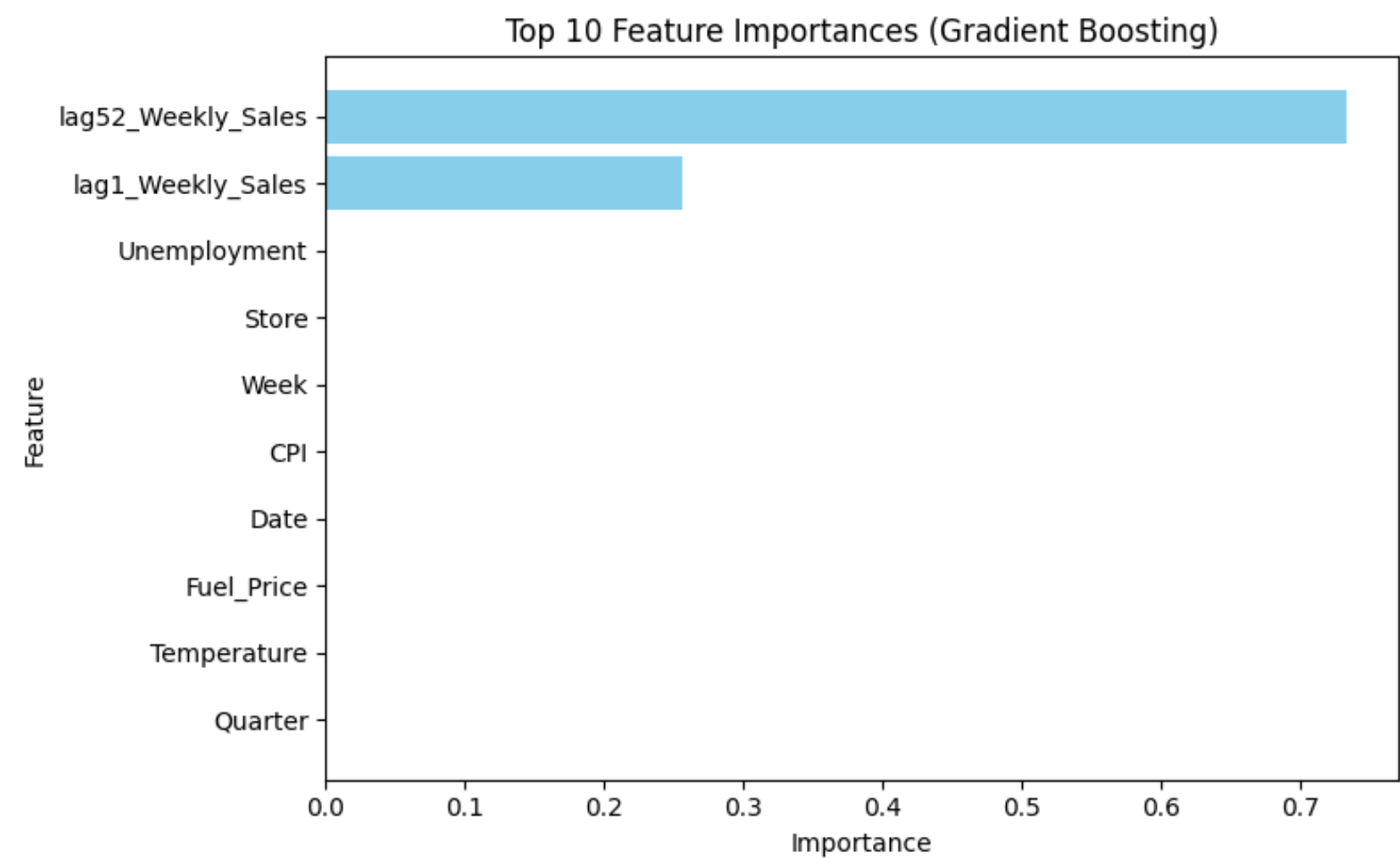
Test

R² = 0.9764
RMSE = 82365.60
MAE = 57947.33
RMSE/Mean = 0.0797

결과 모두 R²가 높고 RMSE/Mean 비율이 낮게 유지되어, GBR 모델이 과적합 없이 견고한 성능을 보여줌

GBM

[변수 중요도]



Feature Importance	Importance
lag52_Weekly_Sales	0.734248
lag1_Weekly_Sales	0.256636
Unemployment	0.001560
Store	0.001555
Week	0.001375
CPI	0.001315
Date	0.001098
Fuel_Price	0.000729
Temperature	0.000497
Quarter	0.000408
Month	0.000332
Holiday_Flag	0.000246
Year	0.000000
DayOfWeek	0.000000

LightGBM & XGBoost

[모델 학습 및 성능 평가]

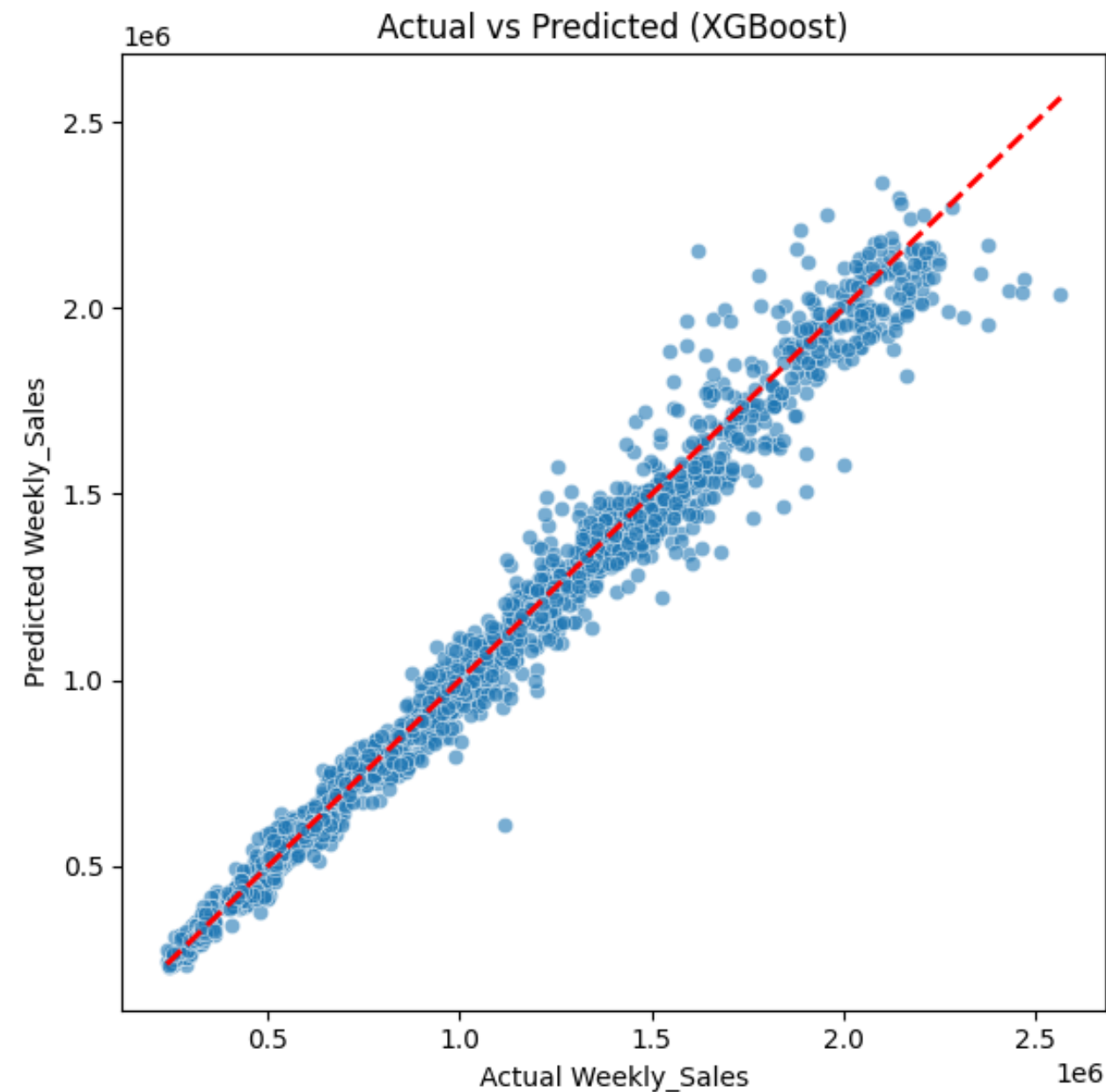
```
# 모델 정의
models = {
    "GradientBoosting": GradientBoostingRegressor(random_state=42, **gbr_params),
    "LightGBM": LGBMRegressor(random_state=42, n_estimators=300, learning_rate=0.1, max_depth=5, subsample=0.8),
    "XGBoost": XGBRegressor(random_state=42, n_estimators=300, learning_rate=0.1, max_depth=5, subsample=0.8)
}

# 모델 학습 및 성능 평가
results = []
for name, model in models.items():
    pipe = Pipeline([
        ("preprocessor", preprocessor),
        ("model", model)
    ])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    r2 = r2_score(y_test, y_pred)
    rmse_val = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    results.append({"Model": name, "R2": r2, "RMSE": rmse_val, "MAE": mae})
```

	R ²	RMSE	MAE
XGBoost	0.978737	78234.020245	50702.848040
Lighst GBM	0.978672	78352.651124	51094.477160
GradientBoosting	0.976432	82365.604419	57947.330603

XGBoost

[실제값 vs 예측값]

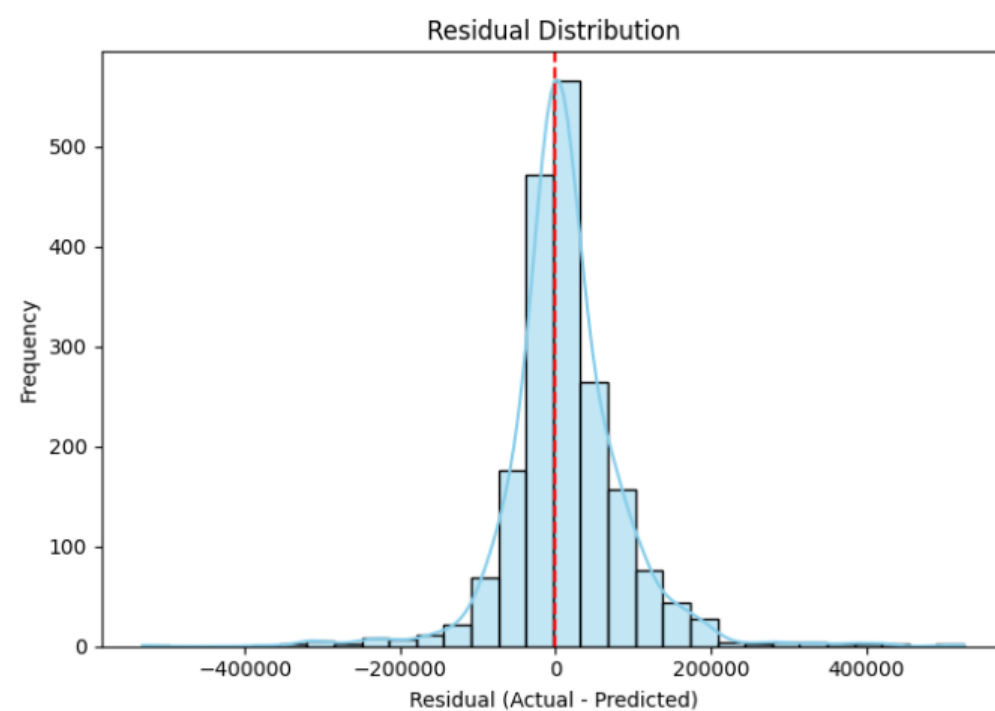


```
# 실제값 vs 예측값
plt.figure(figsize=(6,6))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Weekly_Sales")
plt.ylabel("Predicted Weekly_Sales")
plt.title("Actual vs Predicted (XGBoost)")
plt.tight_layout()
plt.show()
```

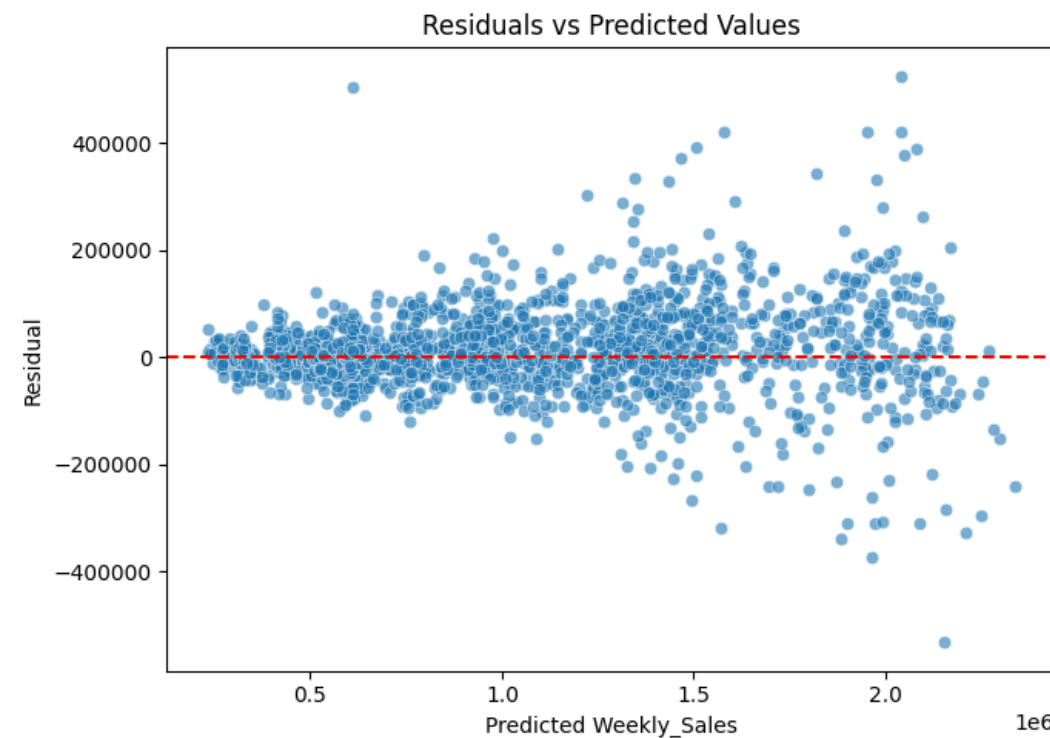
전체 구간에서 예측값이 실제 매출과 선형적으로 잘 맞아떨어지며, 잔차가 크지 않아 안정적인 예측력을 보여줌

XGBoost

[Residual Summary Statistics]



잔차 분포 히스토그램



잔차 vs 예측값

잔차의 등분산성이 깨졌으므로
Box-Cox 변환 사용

Residual Summary Statistics

count 1935.000000

mean 13360.119755

std 77104.744008

min -531692.420000

25% -20903.840000

50% 7065.416250

75% 42473.423750

max 525944.920000

Name: Weekly_Sales, dtype: float64

XGBoost

[모델 학습 및 성능 평가]

Box-Cox 변환 λ : 0.2708

```
# XGBoost 학습 (Box-Cox 변환된 y 사용)
xgb_bc = XGBRegressor(
    random_state=42,
    n_estimators=300,
    learning_rate=0.1,
    max_depth=5,
    subsample=0.8
)
xgb_bc.fit(X_train_proc, y_train_bc)

# 예측 및 역변환
y_pred_bc = xgb_bc.predict(X_test_proc)
y_pred_back_bc = inv_boxcox(y_pred_bc, fitted_lambda) - 1

# 모델 성능 평가
r2_bc = r2_score(y_test, y_pred_back_bc)
rmse_bc = np.sqrt(mean_squared_error(y_test, y_pred_back_bc))
mae_bc = mean_absolute_error(y_test, y_pred_back_bc)

print("\n Box-Cox 변환 후 XGBoost 모델 성능")
print(f"R² (결정계수): {r2_bc:.4f}")
print(f"RMSE (평균 제곱근 오차): {rmse_bc:,.2f}")
print(f"MAE (평균 절대 오차): {mae_bc:,.2f}")
print(f"RMSE / Mean Sales: {rmse_bc / np.mean(y_test):.4f}")
```

Box-Cox 변환 후 XGBoost 모델 성능

R² (결정계수): 0.9787

RMSE (평균 제곱근 오차): 78,211.01

MAE (평균 절대 오차): 52,457.96

RMSE / Mean Sales: 0.0757

모델의 예측 정확도가 향상되고 오차 비율도 약 7%로
안정적으로 유지

XGBoost

[잔차 및 이분산성 검정]

```
# 잔차 및 이분산성 검정
residuals_bc = y_test - y_pred_back_bc
predicted_bc = y_pred_back_bc

X_for_bp = sm.add_constant(predicted_bc)
bp_test_bc = het_breuschpagan(residuals_bc, X_for_bp)
labels = ['Lagrange multiplier statistic', 'p-value', 'f-value', 'f p-value']
bp_results_bc = dict(zip(labels, bp_test_bc))

print("\n Breusch-Pagan Test (After Box-Cox Transform)")
for k, v in bp_results_bc.items():
    print(f"{k}: {v:.4f}")
```

Breusch-Pagan Test (After Box-Cox Transform)

Lagrange multiplier statistic: 175.5274

p-value: 0.0000

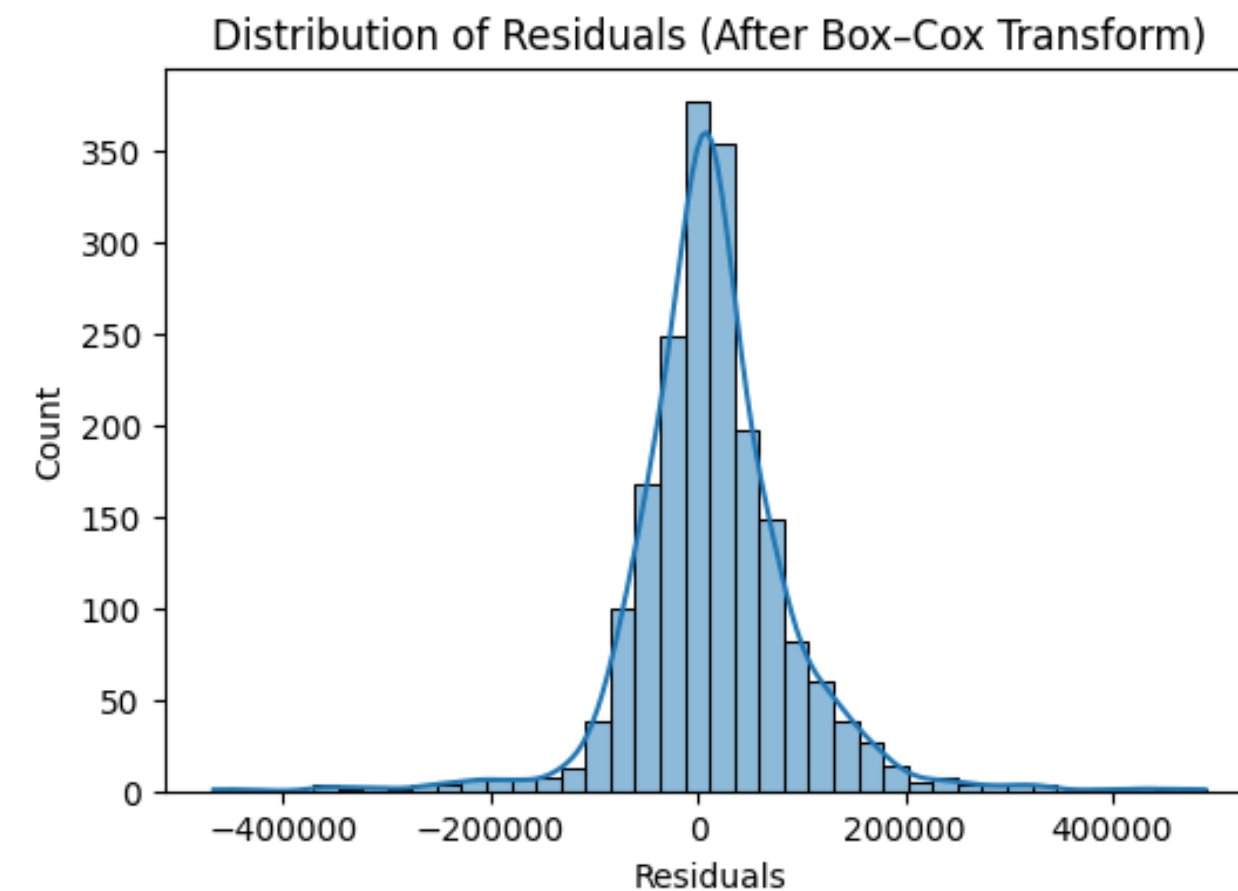
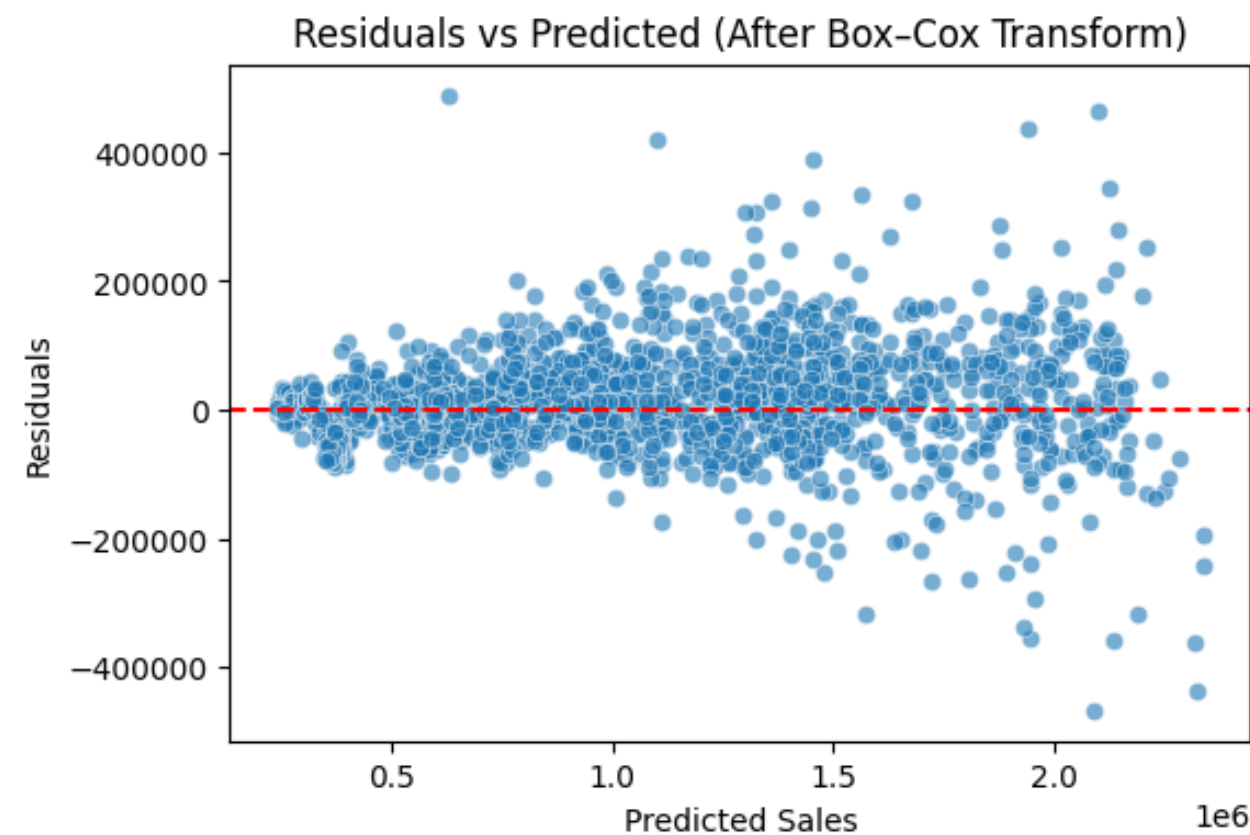
f-value: 192.8387

f p-value: 0.0000

Box-Cox 변환을 사용하였음에도,
유의미한 이분산성이 존재하는 것으로 나타남

XGBoost

[잔차 시각화]



잔차의 폭이 커지는 모습이 보여 등분산성은 완전히 충족되지 않지만,
변환하기 전보다는 완화된 것으로 보임
잔차 분포 자체는 비교적 정규 형태를 유지

5. 결론

결론

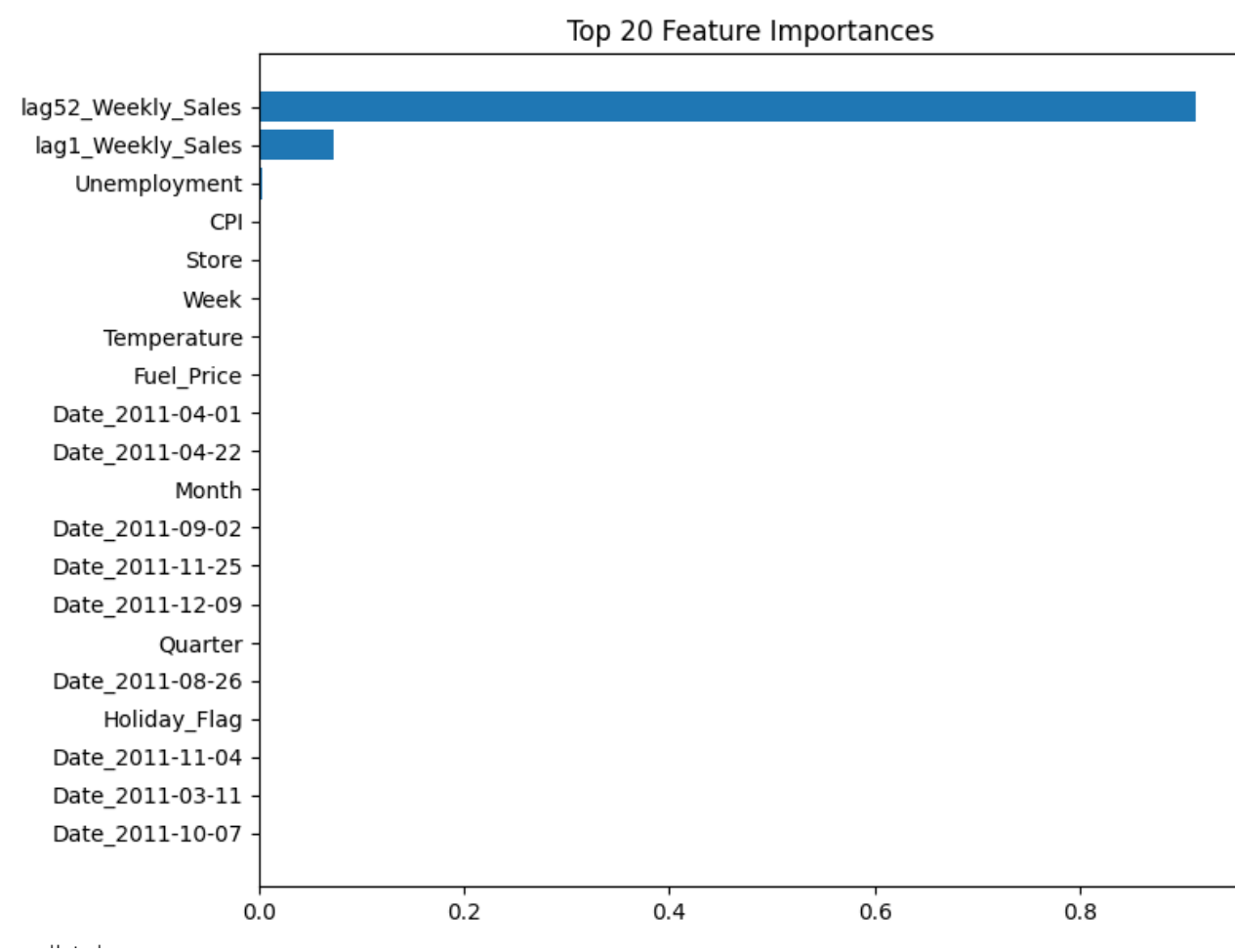
[Linear Regression]

feature	OLS_Scaled_Coef	Ridge_Coef	Lasso_Coef
Temperature	-0.000074	-0.000744	-0.000000
Fuel_Price	-0.007029	-0.006249	-0.004947
CPI	-0.000650	-0.000663	0.000000
Unempolymnt	-0.003123	-0.003249	-0.001437
lag1_log_Weekly_Sales	0.201534	0.221121	0.201624
lag52_log_Weekly_Sales	0.393663	0.370746	0.391585

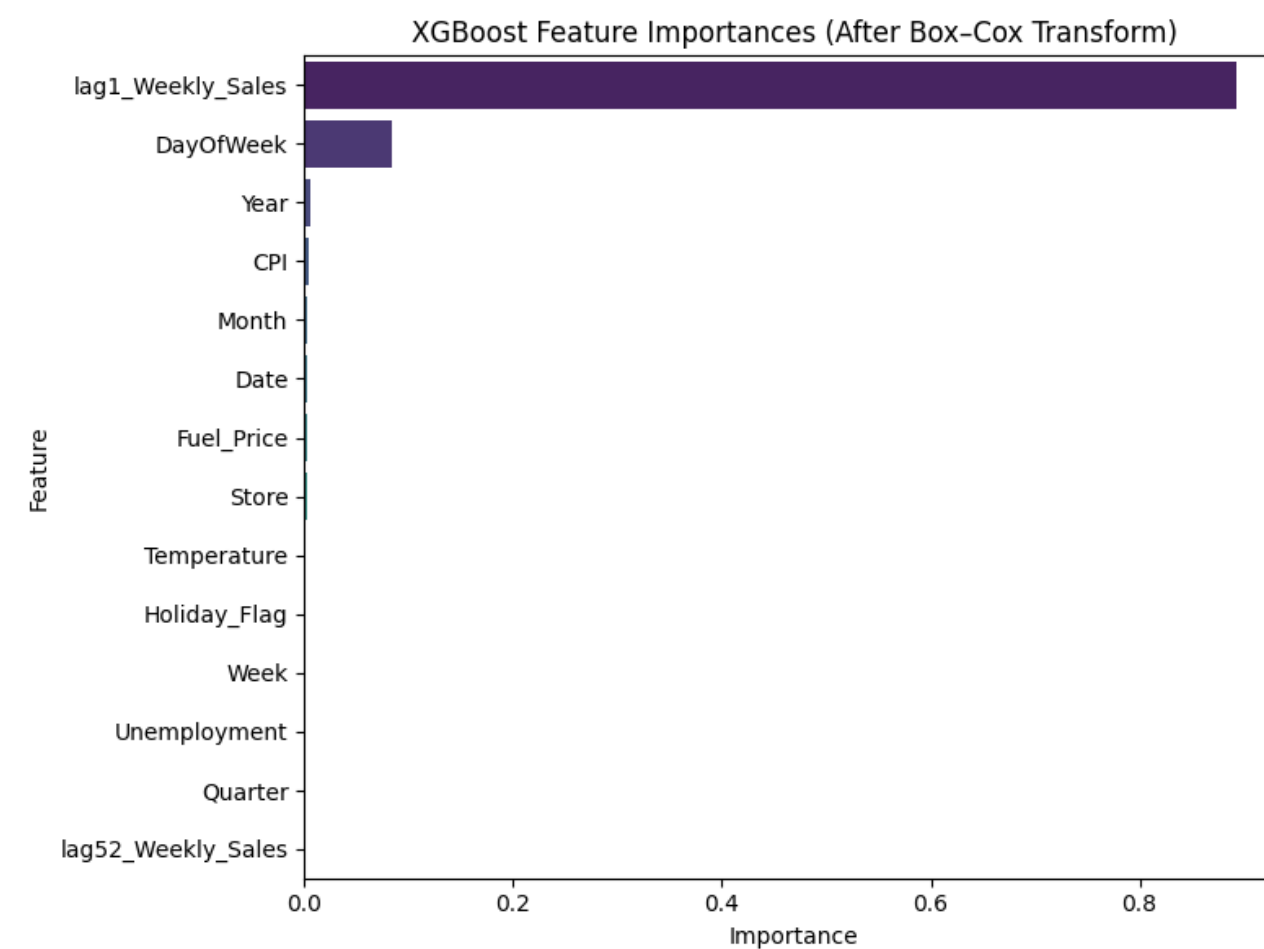
- 세 모델 모두 lag1과 lag52의 회귀계수가 다른 독립변수들의 회귀계수보다 압도적으로 커서, 해당 변수들의 영향을 많이 받음.
- 다시 말하면, 다른 변수들은 매출에 크게 영향을 못 미치고 있음.
- 컬럼이 더 많은 데이터셋으로 후속 연구 시 더 다양한 결과가 예상됨.

결론

[RandomForest 변수 중요도]



[XGBoost 변수 중요도]



Q&A

감사합니다
